Javascript

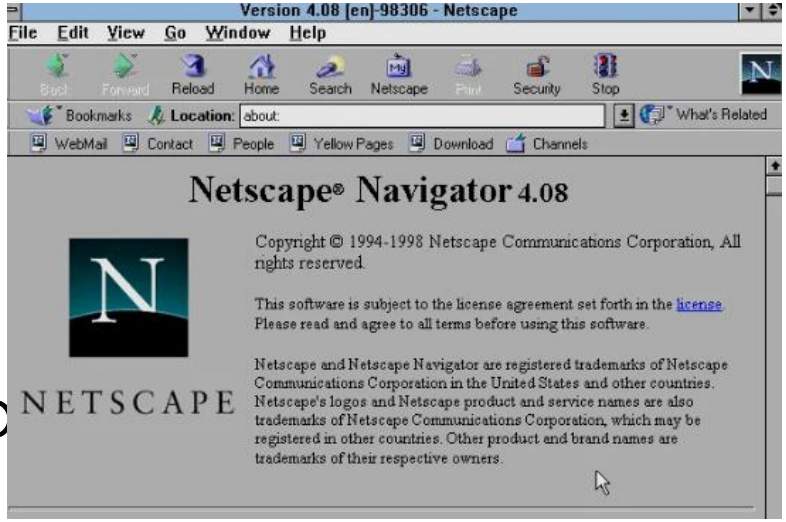
MBP 214 Web Programlama

Bahar 25-26

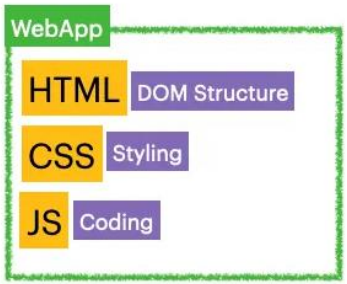
İEU Bilgisayar Prog.

JavaScript

- Javascript , Netscape firması tarafından C dilinden esinlenilerek yazılmıştır.
  LiveScript & Netscape Navigator
- Yazılma amacı Html'in sahip olmadığı bazı özelliklerin web sayfalarında kullanılmak istenmesidir
- Java programlama dilinden tamamen farklıdır karıştırmayınız. JavaScript sadece isim babası



JavaScript Amacı

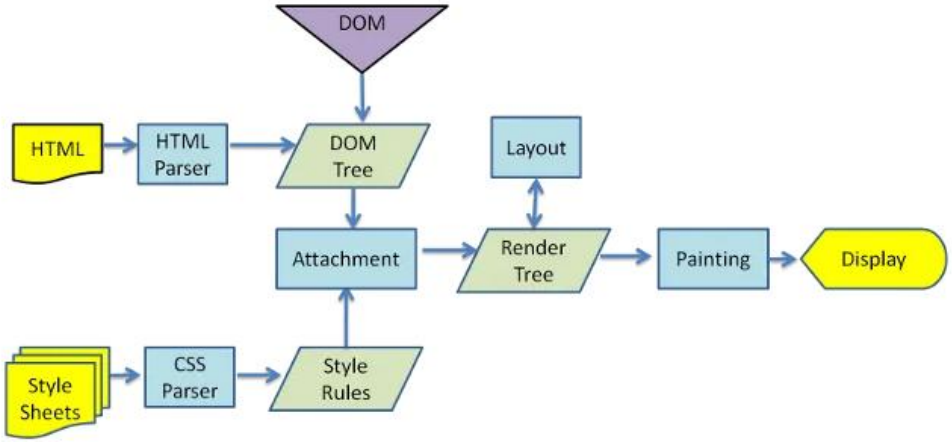


JavaScript

Web Sayfası = İçerik (HTML) + Sunum (CSS) + Etkileşim (JS)

- **Etkileşim:** Kullanıcı bir butona tıkladığında formun onaylanması, açılır menülerin çalışması veya bir "beğen" butonunun anında renk değiştirmesi JS ile olur.
- **Dinamik İçerik:** Sayfayı yenilemeden verilerin güncellenmesini sağlar (örneğin, canlı skorlar veya mesajlaşma uygulamaları).
- **Hız:** İşlemlerin çoğu kullanıcının bilgisayarında (istemci tarafında) gerçekleştiği için sunucu yükünü azaltır ve hızı artırır.
- **Modern API'ler:** Coğrafi konum (GPS), kamera erişimi ve ses işleme gibi gelişmiş özellikleri tarayıcıya taşır.

Web Tarayıcıda Nasıl Render Edilir?



1. **Ayrıştırma (Parsing):** Tarayıcı HTML'i okuyarak DOM ağacını, CSS'i okuyarak CSSOM ağacını oluşturur.
2. **JS Engine Devreye Girer:** Tarayıcının içindeki JavaScript motoru (örneğin Chrome'daki V8 motoru) kodu satır satır okur.
3. **Engelleme (Blocking):** Varsayılan olarak, tarayıcı bir <script> etiketiyle karşılaştığında HTML'i okumayı durdurur, JS'yi indirir ve çalıştırır. (*Bu yüzden performans için JS dosyaları genellikle sayfanın en altına konur*).
4. **İşleme Ağacı (Render Tree):** DOM ve CSSOM birleşerek "Render Tree"yi oluşturur. JS bu ağaç üzerinde bir değişiklik yaparsa (örneğin bir elementi gizlerse), tarayıcı sayfayı yeniden çizer (Repaint veya Reflow).

Nasıl Kullanılır?

Dahili (internal)

- <html> içinde <script> taglerine yazılarak

Harici (external)

- .js uzantılı bir dosya oluşturup HTML'e bağlanarak.

<script src="script.js"></script>

Harici Kullanım - JavaScript Kodlarını Kaynak Dosyada Saklamanın Avantajları

- **Kolay Bakım ve Güncelleme**: Eğer aynı kodlar ana sayfa ve tüm alt sayfalarda kullanılıyorsa, kaynak dosya kullanmadığınızda bir değişiklik yapmak için her sayfadaki kodu tek tek düzenlemeniz gerekir. Ancak harici bir dosya kullanırsanız, sadece bu tek dosyayı değiştirmeniz yeterli olur.
- **Zaman Tasarrufu**: Değişiklik ihtiyacı duyulduğunda her sayfayı ayrı ayrı düzenlemek çok fazla zaman alacaktır. Harici dosya yapısı bu süreci hızlandırır.
- **Kod Düzeni (Temiz Kod):** Projenizdeki kod karmaşasından kurtulmanızı sağlar ve HTML yapınızın daha okunabilir kalmasına yardımcı olur.
- **Hata Tespiti:** Hataları ayıklamak ve tespit etmek merkezi bir dosya üzerinden çok daha kolaydır.
- **src Özniteliği Kullanımı**: Harici dosyalar her sayfaya <script src="..."> özniteliği kullanılarak kolayca dahil edilebilir.

Javascript

- Javascript kodlarını yazmak için Windows kullanıcıları için NotePad yeterlidir.
- Genellikle yazım tarzı şu şekildedir:

```
<script>
JavaScript kodlar
</script>
```

- Kullanılacak yere göre Html'in içerisinde kullanılır. Fakat genelde <head>...</*head*> etiketleri arasında kullanılır.
- Javascript kodları bittiğinde elinizde asla kendi başına çalışan uzantısı exe veya com olan bir dosya olmaz.
- Her zaman için tarayıcı tarafından yorumlanması gerekir. Yorumlanması demek Javascript kodunun çalışması anlamındadır.

<script language="javascript" src="“ornek.js" type="text/javascript"> </script>

JavaScript Söz Dizimi Kuralları

JavaScript'in söz dizimi C, C# ve Java dillerine benzerlik gösterir.

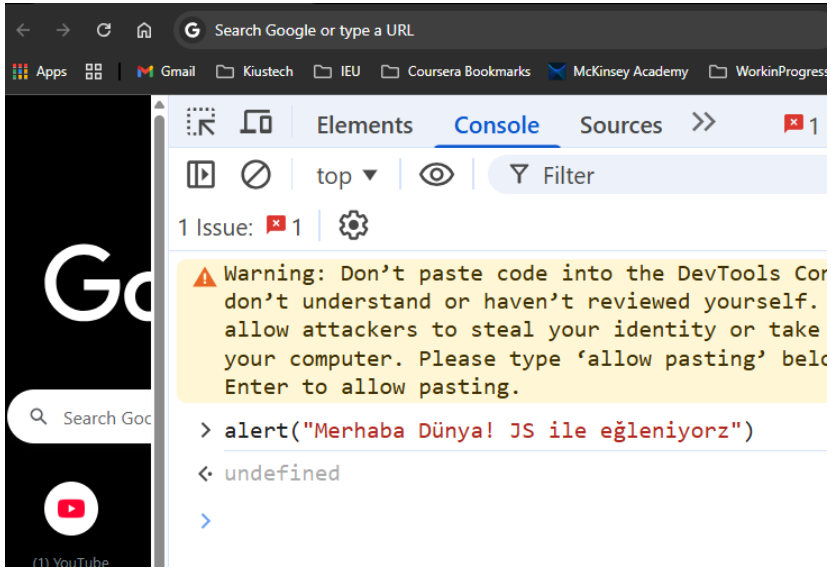
- **Noktalı Virgül (;):** JavaScript'te her bir komut satırı (deyim) ; ile bitmelidir. Örneğin: var i = 10;.
- **Büyük-Küçük Harf Duyarlılığı:** Dil yapısı gereği büyük ve küçük harfler birbirinden farklı algılanır.
- **Nokta (.) Operatörü:** Bir nesnenin özelliklerine veya metotlarına erişmek için kullanılır.
- **Açıklama Satırları (Comments):** Tarayıcı tarafından işlenmeyen, sadece geliştiriciye not bırakmak için kullanılan bölümlerdir.
  - **Tek satırlı açıklama:** // karakteri ile başlar.
  - **Çok satırlı açıklama:** /* ile başlar ve */ ile biter.

```
var araba = {
    marka: "Tesla",
    model: "Model 3",
    yil: 2024
};

// Nokta operatörü ile verilere erişim:
console.log(araba.marka); // Çıktı: Tesla
console.log(araba.yil);   // Çıktı: 2024
```

JavaScript Nerede Çalışır? (Mutfak: Browser Console)

- JavaScript'in en güzel yanı, hiçbir program kurmadan doğrudan **Tarayıcıda (Browser)** çalışmasıdır.
- Tarayıcı >> F12 (İncele) >> Network altında yüklene dosyaları inceleyerek JS dosyasını görüntüleyebilirsiniz.

```
alert("Merhaba Dünya! Ben artık bir programcıyım.");
```



Javascript-Alert İkaz Metodu

- Alert yöntemi, kullanıcıya bir uyarı mesajı ve bir OK düğmesi görüntülemek için kullanılır

```
alert ("Merhaba");
```

Javascript-Confirm Metodu

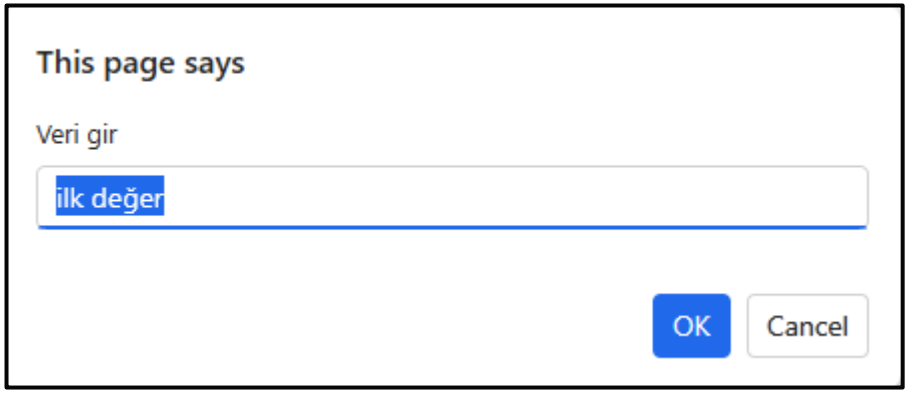
- Confirm yöntemi OK ve Cancel düğmelerine sahip bir mesaj görüntülemek için kullanılır. Ok basarsa True, Cancel' a basılırsa false döndürür.

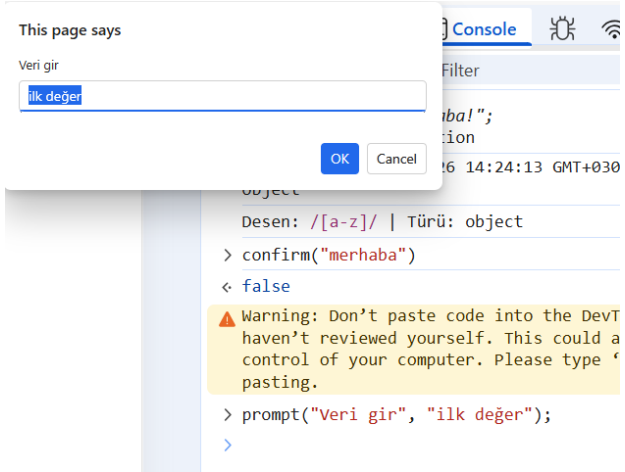
```
confirm("Onayliyormusunuz?")
```

Javascript-Prompt Metodu

- Kullanıcı girdilerini almak için bir iletişim kutusunu görüntüler.

```
prompt ("Veri gir","ilk deger");
```

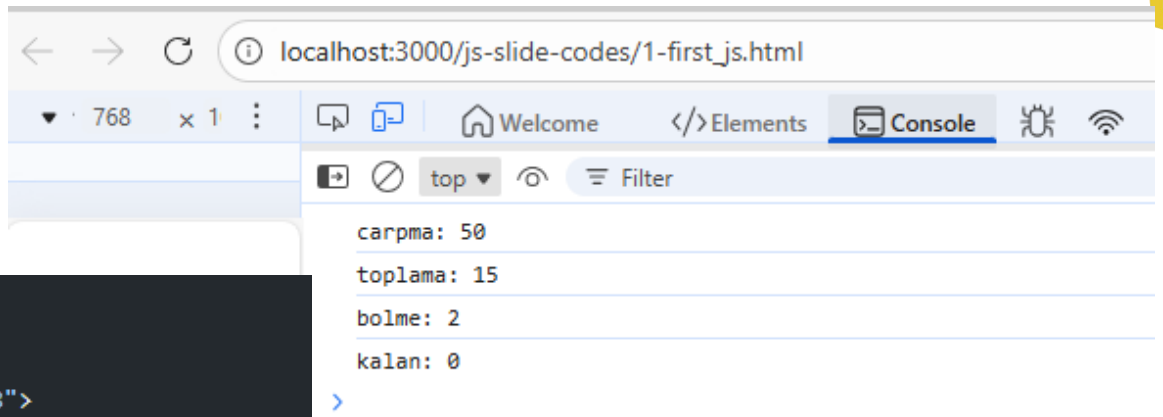




JavaScript

js-slide-codes >> 1-first_js.html

İlk Kod - Dahili




```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Konsoldan çıktı almak</title>
    <script type="text/javascript">
        var i = 10;
        var j = 5;

        var carpma = i * j;
        var toplama = i + j;
        var bolme = i / j;
        var kalan = i % j;

        console.log("carpma: " + carpma);
        console.log("toplama: " + toplama);
        console.log("bolme: " + bolme);
        console.log("kalan: " + kalan);
    </script>
</head>
<body>

</body>
</html>
```

Kodun Analizi

- **Değişken Tanımlama:** i değişkenine 10, j değişkenine ise 5 değeri atanmış.
- **Matematiksel İşlemler:**
  - **Çarpma (*):** $10 \times 5 = 50$
  - **Toplama (+):** $10 + 5 = 15$
  - **Bölme (/):** $10 / 5 = 2$
  - **Mod Alma (%):** 10'un 5'e bölümünden kalan 0
- **Konsol Çıktısı:** console.log() komutu, yanındaki metinle hesaplanan sonucu birleştirerek tarayıcının geliştirici araçları kısmına yazdırır.

KOD: js-react-ders-sunum-kodlari >> 1-ilk_kod.html

İlk Kod - Harici

JavaScript

```javascript
var i = 10;
var j = 5;

var carpma = i * j;
var toplama = i + j;
var bolme = i / j;
var kalan = i % j;

console.log("carpma: " + carpma);
console.log("toplama: " + toplama);
console.log("bolme: " + bolme);
console.log("kalan: " + kalan);
```

HTML

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Konsoldan çıktı almak</title>
    <script type="text/javascript" src="hesaplama.js"></script>
</head>
<body>
</body>
</html>
```

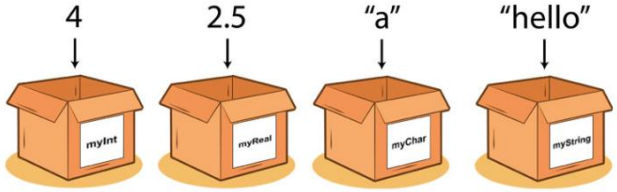
KOD: js-react-ders-sunum-kodlari>> 2-ilk_kod-harici.html

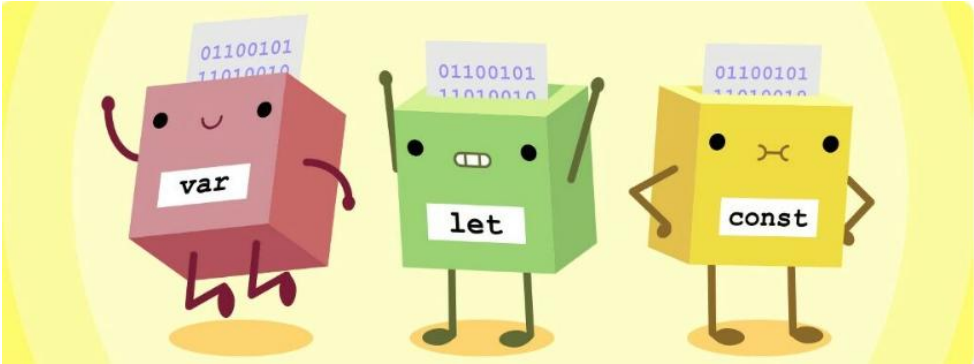
JS Temelleri

- Değişkenler
  - İsimlendirme
  - Kapsam
- Veri Türleri
- Tür Dönüşümleri
- Operatörler

Değişkenler:
Verinin "Saklama Kapları"

- Bilgisayarın hafızasında (RAM) bir yer ayırıp içine bir isim verip değer koymaktır.
- Bir taşınma kolisi gibi düşünün;
  - üzerine "Mutfak Eşyaları" yazarsınız (Değişken adı) ve
  - içine tabakları koyarsınız (Değişken değeri).

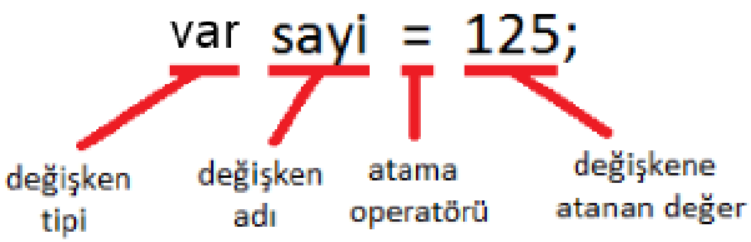




- **var**, değişken tanımlamak için kullanılan **eski** bir yöntemdir.
- **let**, değişken tanımlamak için kullanılan **yeni** bir yöntemdir.
- **const**, değişken değerinin sabit olduğunu belirtir ve herhangi bir **değişikliğe izin vermez**.

Değişken Adı Kuralları

```
var title = 'Code Life';
let publisher = 'House of Books';
const author = 'Melvin Gray';
```



- Değişken isimleri Türkçe karakter ***içermemelidir.***
- Değişken isimleri büyük ve küçük harf ***duyarlıdır.***
- Değişken isimlerinde ilk karakter ***bir sayı olamaz.***
- Değişken isimlerinde JavaScript etiketleri ***kullanılamaz.***
- Değişken isimlerinde sayı, harf, alt çizgi ve dolar işareti ***kullanılabilir;***
- Boşluk, noktalama işareti veya sembol ***kullanılamaz.***

JavaScript

Değişken isimlerinde JavaScript etiketleri kullanılamaz.

- break
- case
- catch
- continue
- debugger
- default
- delete
- do
- else
- finally
- new
- return
- switch
- for
- typeof
- function
- if
- in
- this
- throw
- try
- var
- void
- while
- with
- instanceof

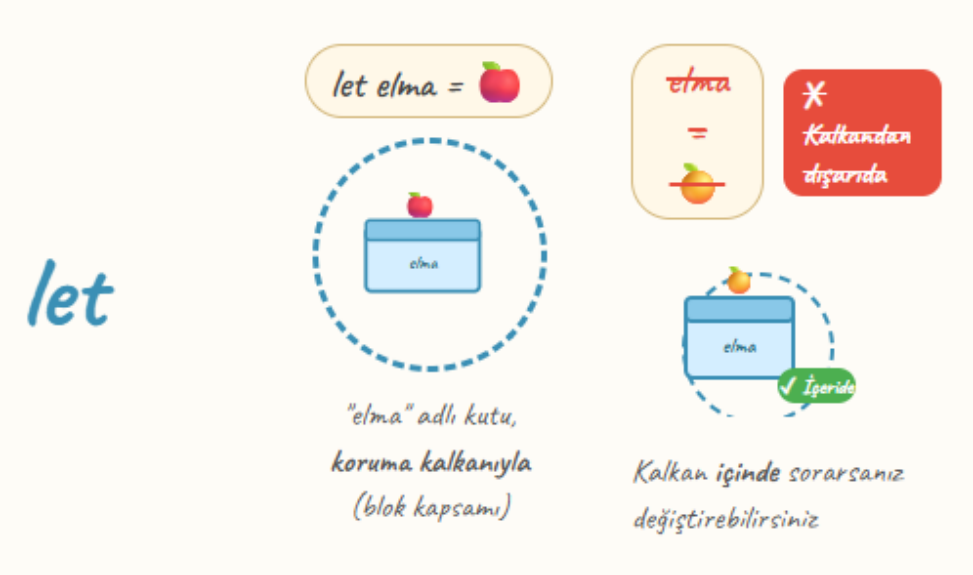
var

- JS'in ilk yıllarından beri var olan emektar komut.
- Dikkat edilmesi gereken bir tür!!!
- Bir değişkeni var ile tanımlarsanız, aynı isimle tekrar tanımlamanıza izin verir.
- **Kapsam (Scope):** var biraz gezentidir; bir fonksiyonun içinde değilse her yerden erişilebilir (Global Scope).

KOD: js-react-ders-sunum-kodlari>> 3-degiskenler.html

let

- Modern JavaScript (ES6) ile gelen, günümüzde en çok kullanılan değişken tanımlama yöntemidir.
- Aynı isimle ikinci kez tanımlama yapmanıza izin vermez, hata fırlatır.
- **Kapsam (Scope):** Sadece bulunduğu süslü parantezlerin { } içinde yaşar. Odadan dışarı çıkmaz.

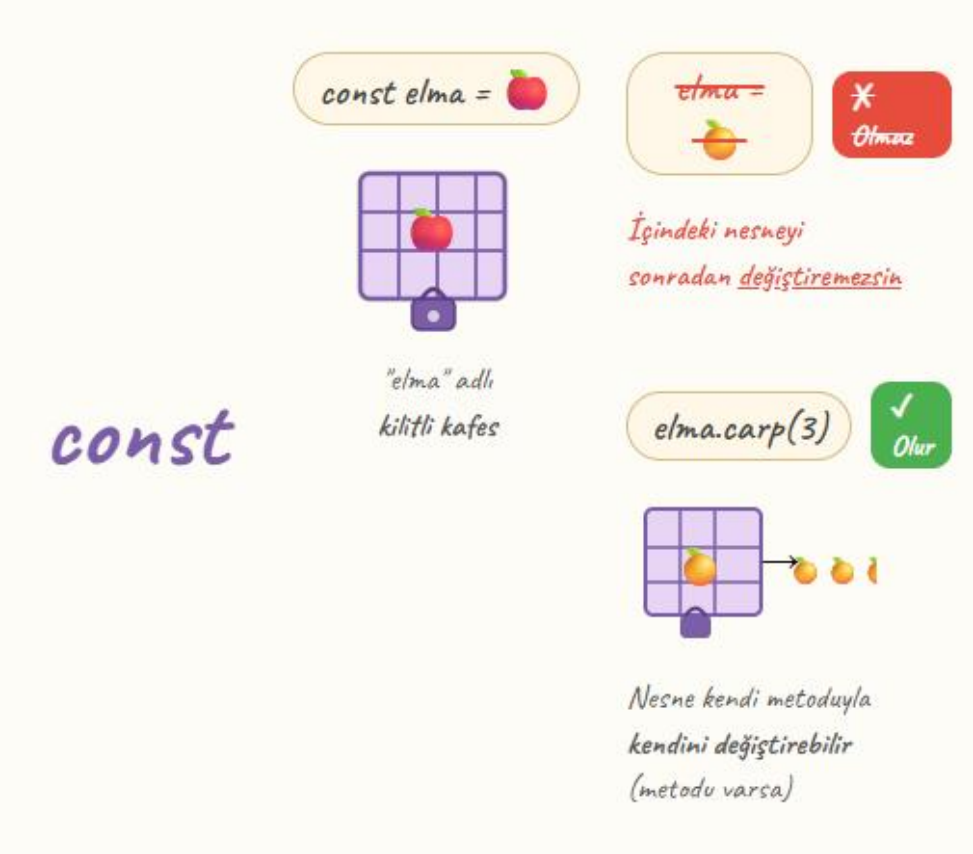


KOD: js-react-ders-sunum-kodlari>> 3-degiskenler.html

JavaScript

const

- Asla değişmez
- İngilizce "Constant" (Sabit) kelimesinden gelir.
- **Özelliği:** Bir kez değer atadınız mı, bir daha asla değiştiremezsiniz.
- **Kullanım Yeri:** Pi sayısı gibi sabitler, veritabanı bağlantı bilgileri gibi değişmesini istemediğimiz her yer.



KOD: js-react-ders-sunum-kodlari>> 3-degiskenler.html

var – let – const karşılaştırma

- **var**, değişken tanımlamak için kullanılan **eski** bir yöntemdir.
- **let**, değişken tanımlamak için kullanılan **yeni** bir yöntemdir.
- **const**, değişken değerinin sabit olduğunu belirtir ve herhangi bir **değişikliğe izin vermez**.

Özellik	`var`	`let`	`const`
Tekrar Tanımlama	Evet (Tehlikeli!)	Hayır	Hayır
Değer Güncelleme	Evet	Evet	Hayır
Modernlik	Eski (Kullanma)	Yeni (Standart)	Yeni (Sabitler İçin)

Dikkat!!! - "Hata Yaparsan Yanarsın"
Köşesi: Hoisting (Yukarı Çekme)

JavaScript

- **var** ile tanımlanan bir değişken, siz onu daha yazmadan kodun en başında varmış gibi davranabilir (Hoisting).
  - Ama **let** ve **const** duyarlı ve anlayışlıdır; onları yazmadan onlara seslenirseniz size "Tanımıyorum ki!" derler.
- **let** ve **const** kullanın!
  - var kapsamı çok geniş
  - var Yeniden değer verilmesi kontrolü zorlu
  - var Hoisting

KOD: js-react-ders-sunum-kodlari >> 3-degiskenler.html

JavaScript

VAR / LET Kıyaslaması

Özellik	var Anahtar Kelimesi	let Anahtar Kelimesi
Kapsam (Scope)	Fonksiyon kapsamlıdır (Function-scoped).	Blok kapsamlıdır (Block-scoped).
Yeniden Tanımlama	Aynı isimle tekrar tanımlamaya izin verir.	Aynı alan içinde tekrar tanımlamaya izin verilmez.
Başlatma (Initialization)	Başlangıç değeri atanmadan tanımlanabilir.	Kullanılmadan önce mutlaka başlatılması gerekir.
Döngü Kullanımı	Tüm fonksiyon boyunca paylaşılır.	Tanımlandığı blok ile sınırlandırılır.
Hoisting (Yukarı Taşıma)	Yukarı taşınır (Kapsamının en üstüne taşınmış gibi davranır).	Yukarı taşınmaz (Tanımlanana kadar "geçici ölü bölge" - temporal dead zone içinde kalır).
JavaScript Versiyonu	Bir ECMAScript1 özelliğidir.	ECMAScript6 (ES6) ile tanıtılmıştır.

Koddaki Hata nedir?

- const ile tanımlanan değer değiştirilemez.
- Değişken isimleri rakamla başlayamaz.
- String (metin) ifadeler tırnak içine alınmalıdır ("Merhaba...").

```
const okulAdi = "Ege MYO";
okulAdi = "Dokuz Eylül MYO";

let 1.Sira = "Ahmet";

var mesaj = Merhaba JavaScript;
```

MİNİ TEST – soru 1

JavaScript'te bir değişkenin değerinin program boyunca hiç değişmemesini istiyorsak hangi anahtar kelimeyi kullanmalıyız?

A. var

B. let

C. fixed

D. const

JavaScript

MİNİ TEST – soru 2

Örnekte geçen `let dersler = ["Programlama", "Veritabanı"];` tanımlaması hangi veri tipine örnektir?

A. Boolean

B. Object

C. Array (Dizi)

D. String

MİNİ TEST – soru 3

JavaScript

Bir öğrencinin mezun olup olmadığını belirten `let mezunMu = false;` değişkeni hangi veri tipindedir?

A. Null

B. Boolean

C. Number

D. Undefined

MİNİ TEST – soru 4

Aşağıdaki değişken tanımlamalarından hangisi JavaScript yazım kurallarına (syntax) göre hatalıdır?

A. let kullanici1 = "Veli";

B. let 1.kullanici = "Ali";

C. let $fiyat = 50;

D. let _sayi = 10;

MİNİ TEST – soru 5**

Bir değişken tanımlandığı halde ona henüz bir değer atanmamışsa, bu değişkenin otomatik olarak aldığı veri tipi hangisidir?

A. 0

B. null

C. empty

D. undefined

MİNİ TEST – soru 6**

Aşağıdaki kod parçasının çıktısı ne olur?

```
let x = "5";
let y = 2;
console.log(x + y);
```

A. 7

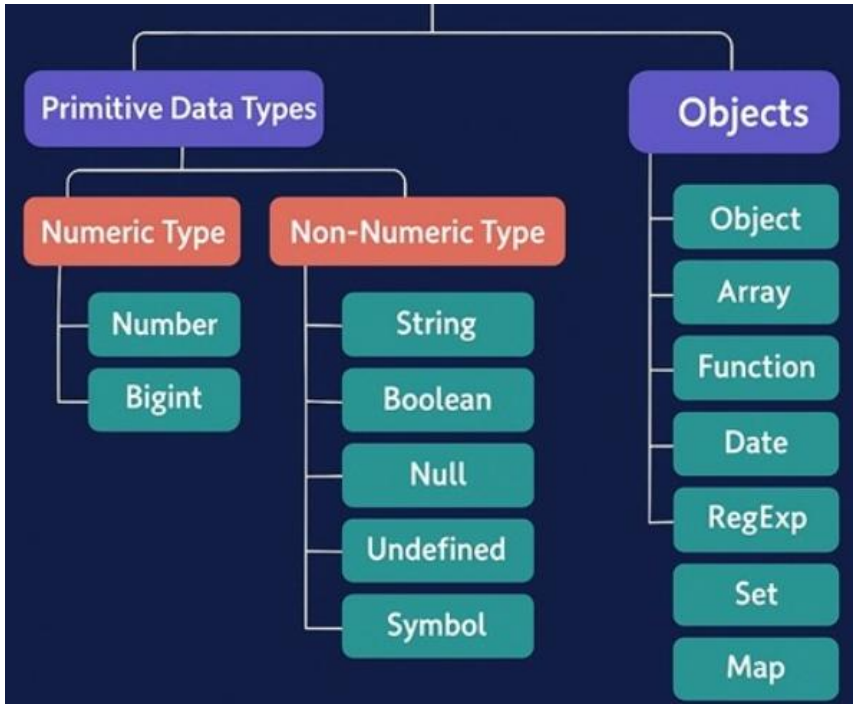
B. Hata verir

C. "52"

D. NaN

Veri Tipleri: "Kutunun İçinde Ne Var?"

- JavaScript'te veri tipleri, kolinin içine koyduğumuz eşyanın **cinsi** gibidir.
- JavaScript "dinamik" bir dildir.
  - **Yani bir değişkenin içine önce bir sayı, sonra bir metin koymanıza pek kızmaz (ama biz yine de düzenli olalım!).**



Veri Tipleri

İlkel Veri Tipleri (Primitive Data Types)

Tek bir değer tutarlar ve değiştirilemezler (immutability).

- **Sayısal Tipler (Numeric)**
  - **Number:** Her türlü sayı. 10, 3.14, -5. JavaScript'te tam sayı/ondalıklı sayı ayrımı yoktur, hepsi "Number"dır.
  - **BigInt:** Çok ama çok büyük sayılar için kullanılır. Normal Number tipinin kapasitesini aşan (2^53'ten büyük) sayılarda sonuna n ekleyerek kullanılır.
- **Sayısal Olmayan Tipler (Non-Numeric)**
  - **String:** Metinler. Tırnak içindeki her şey: "Merhaba Dünya".
  - **Boolean:** Mantıksal anahtarlar. Sadece true veya false.
  - **Null:** "Bilerek boş bırakılmış" demektir. Bir değişkenin değerinin boş olduğunu kasten belirtmek için biz atarız.
  - **Undefined:** "Henüz tanımlanmamış" demektir. Bir değişken oluşturulmuş ama içine değer konmamışsa JS otomatik olarak bunu atar.

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

Nesneler (Objects) –

İçlerinde birden fazla değer ve hatta fonksiyon barındırabilirler.

- **Object (Nesne):** { anahtar: değer } şeklinde çalışan temel yapı.
  - { ad: "Volkan", yas: 40 }.
- **Array (Diziler):** Listelerdir. Köşeli parantez içinde tutulur.
  - ["Elma", "Armut", "Muz"].
- **Function (Fonksiyonlar):** Aslında JS'de fonksiyonlar da birer objedir! Kod bloklarını saklar ve istediğimizde çalıştırır.
- **Date:** Tarih ve saat bilgilerini tutan özel nesneler.
- **RegExp:** Düzenli ifadeler. Metinler içinde karmaşık arama ve değiştirme (örn: e-posta kontrolü) yapmak için kullanılır.
- **Set:** İçinde aynı değerden sadece bir tane barındıran özel listeler. (Tekrar eden eleman istemediğimizde kullanırız).
- **Map:** Objeye benzer ama anahtar kısmına sadece metin değil, her şeyi koyabildiğimiz gelişmiş sözlük yapılarıdır.

Veri Tipleri

İlkel Veri Tipleri (Primitive Data Types)

Tek bir değer tutarlar ve değiştirilemezler (immutability).

- Sayısal Tipler (Numeric)
  - **Number:** Her türlü sayı. 10, 3.14, -5. JavaScript'te tam sayı/ondalıklı sayı ayrımı yoktur, hepsi "Number"dır.
  - **BigInt:** Çok ama çok büyük sayılar için kullanılır. Normal Number tipinin kapasitesini aşan (2^53'ten büyük) sayılarda sonuna n ekleyerek kullanılır.
- Sayısal Olmayan Tipler (Non-Numeric)
  - **String:** Metinler. Tırnak içindeki her şey: "Merhaba Dünya".
  - **Boolean:** Mantıksal anahtarlar. Sadece true veya false.
  - **Null:** "Bilerek boş bırakılmış" demektir. Bir değişkenin değerinin boş olduğunu kasten belirtmek için biz atarız.
  - **Undefined:** "Henüz tanımlanmamış" demektir. Bir değişken oluşturulmuş ama içine değer konmamışsa JS otomatik olarak bunu atar.

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

Veri Tipleri

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

Nesneler (Objects) –

İçlerinde birden fazla değer ve hatta fonksiyon barındırabilirler.

- **Object (Nesne):** { anahtar: değer } şeklinde çalışan temel yapı.
  - { ad: "Volkan", yas: 40 }.
- **Array (Diziler):** Listelerdir. Köşeli parantez içinde tutulur.
  - ["Elma", "Armut", "Muz"].
- **Function (Fonksiyonlar):** Aslında JS'de fonksiyonlar da birer objedir! Kod bloklarını saklar ve istediğimizde çalıştırır.
- **Date:** Tarih ve saat bilgilerini tutan özel nesneler.
- **RegExp:** Düzenli ifadeler. Metinler içinde karmaşık arama ve değiştirme (örn: e-posta kontrolü) yapmak için kullanılır.
- **Set:** İçinde aynı değerden sadece bir tane barındıran özel listeler. (Tekrar eden eleman istemediğimizde kullanırız).
- **Map:** Objeye benzer ama anahtar kısmına sadece metin değil, her şeyi koyabildiğimiz gelişmiş sözlük yapılarıdır.

Null vs Undefined

Null

- Bir kutu var, içini özellikle biz boşalttık ve "burası şimdilik boş kalsın" dedik (Kutunun içine "BOŞ" etiketi koyduk). `null`

Undefined

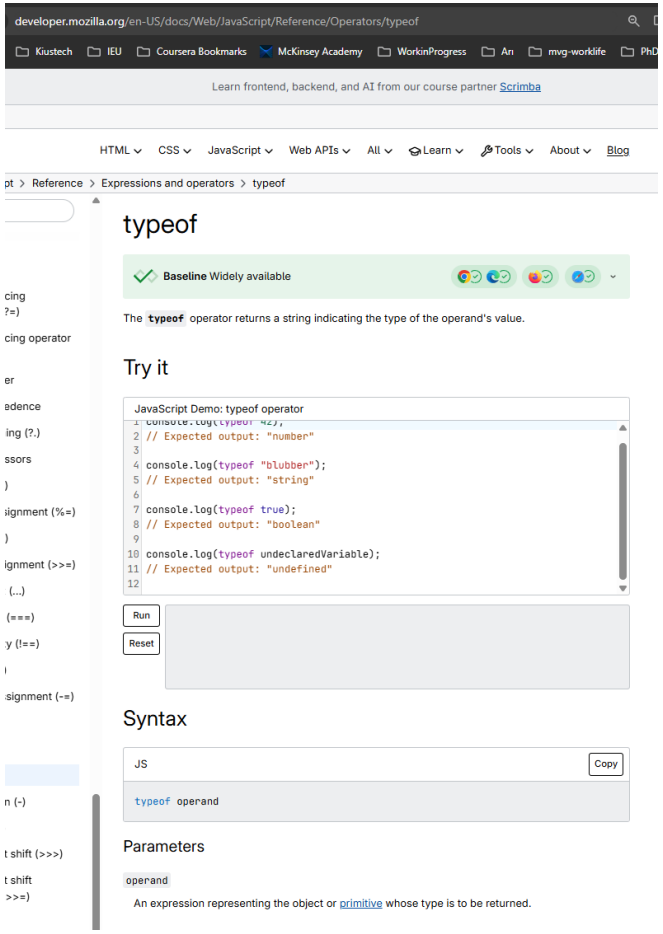
- Bir kutu var ama içi boş, henüz bir şey koymadık (Kutuyu açtın, boş çıktı).

`undefined`

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

Veri Türünü Tespit

- **Typeof()**
  - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/typeof
  - işlenenin (operand'ın) değerinin türünü belirten bir string (metin) döndürür.



KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

ÖDEV 1

SORU 1:

Javascript string literals (kaçış karakterleri – escape characters) nedir ve nasıl kullanılır?

SORU 2:

'…' (tek tırnak), "…" (çift tırnak), `…` (Template Literals) farkı nedir? Örnek ile açıklayınız.

SORU 3:

Number(sayı) veri tipinin max_value ve mi_value değerlerini nasıl elde edersiniz? Bu değerleri tipi sayı olan farklı değişkenlere atayarak "alert" metodu ile yazdırın.

SORU 4: Aşağıdaki kodu JS-HTML ile yazınız:

Adınızı, soyadınızı, yaşınızı, bölümünüzü farklı değişkenlere metin(string) olarak atayın.

Mezuniyet durumunuzu "mezun" isimli bir değişkene mantıksal(Boolean") olarak atayın.

Ekrana

Merhaba, ben …ad soyad…

Yaşım: ……

Bölümüm: ……

Mezunsanız "Mezunoldum!"

Değilseniz "Hala öğrenciyim." olarak yazdırın

İlkel Veri Tipleri (Primitive Data Types) Örnekleri

JavaScript

KOD: js-react-ders-sunum-kodlari
>> 4-veri_turleri.html

```
// --- Sayısal (Numeric) ---
let sayi = 35;
console.log("Değer:", sayi, "| Türü:", typeof sayi); // number

let buyukSayi = 9007199254740991n;
console.log("Değer:", buyukSayi, "| Türü:", typeof buyukSayi); // bigint

// --- Sayısal Olmayan (Non-Numeric) ---
let metin = "İzmir Dokuz Eylül";
console.log("Değer:", metin, "| Türü:", typeof metin); // string

let dogruMu = true;
console.log("Değer:", dogruMu, "| Türü:", typeof dogruMu); // boolean

let bos = null;
console.log("Değer:", bos, "| Türü:", typeof bos); // object* (Dikkat: JS'

let tanimsiz;
console.log("Değer:", tanimsiz, "| Türü:", typeof tanimsiz); // undefined
```

KOD: js-react-ders-sunum-kodlari
>> 4-veri_turleri.html

JavaScript

Nesneler (Objects) Örnekleri

```
// Standart Obje (Anahtar-Değer Çifti)
let hoca = { ad: "Volkan", meslek: "Solution Architect" };
console.log("Obje:", hoca, "| Türü:", typeof hoca); // object

// Array (Dizi / Liste)
let diller = ["JavaScript", "PHP", "Java"];
console.log("Dizi:", diller, "| Türü:", typeof diller); // object (Diziler de

// Function (Fonksiyon)
let selamVer = function() { return "Merhaba!"; };
console.log("Fonksiyon:", selamVer, "| Türü:", typeof selamVer); // function

// Date (Tarih)
let bugun = new Date();
console.log("Tarih:", bugun, "| Türü:", typeof bugun); // object

// RegExp (Düzenli İfade)
let desen = /[a-z]/;
console.log("Desen:", desen, "| Türü:", typeof desen); // object
```

Set ve Map

KOD: js-react-ders-sunum-kodlari >> 4-veri_turleri.html

JavaScript

```
console.log("-----SET-----");
let set = new Set();
set.add("değer1");
set.add("değer2");
set.add("değer3");
console.log("Set:", set, "| Türü:", typeof set);
set.add("değer2"); /* Set'e aynı değeri eklemeye çalıştığımızda,
 zaten var olduğu için eklenmez.*/
console.log("Set (Tekrar ekleme denemesi sonrası):", set);

console.log("-----MAP-----");
let map = new Map();
map.set("key1", "value1");
map.set("key2", "value2");
map.set("key3", "value3");
console.log("Map:", map, "| Türü:", typeof map);
console.log("Map'ten key2'nin değeri:", map.get("key2")); // value2
```

“typeof null” Tuzağı

- JS ilk sürümlerinden kalan bir hata nedeniyle **object** döner. Ama biz biliyoruz ki null aslında bir **Primitive** (İlkel) veri türüdür. Yani bu bir "yazılımsal hata" klasiğidir!

```
let nullDeger = null;
console.log("Değer:", nullDeger, "| Türü:", typeof nullDeger);
```

Uygulamada: js-react-ders-sunum-kodlari >> veri_tipleri_html.html

KOD: js-react-ders-sunum-kodlari>> 4-veri_turleri.html

JavaScript

Sınıf Çalışması

- Kendi isim, soyad, yaş ve öğrenci olup olmadığınızı içeren bir ***ogrenci*** objesi oluşturun ve **console.log** ile yazdır

```
let ogrenci = {
    isim: "Mustafa",
    yas: 20,
    aktifMi: true
};
console.log(ogrenci);
```

```
▼ {isim: 'Mustafa', yas: 20, aktifMi: true} i
    aktifMi: true
    isim: "Mustafa"
    yas: 20
  ▶ [[Prototype]]: Object
```

Tür Dönüşümleri – Dikkat!!!

Açık Dönüşüm (Explicit Conversion)

- **String'e Dönüştürme:** Bir sayıyı veya boolean değeri metne çevirmek için **String()** kullanılır.
- **Sayıya Dönüştürme:** Metin içindeki sayıları matematiksel işleme sokabilmek için **Number()** kullanılır.

Kapalı/Otomatik Dönüşüm (Implicit Conversion)

- JavaScript'te program kodlarının akışında değişkenin veri türü, içerisine atanan değere göre örtülü olarak (*otomatik olarak*) değişebilir
- Type Coercion

Tür Dönüşümleri - Açık Dönüşüm

JavaScript

Açık Dönüşüm (Explicit Conversion)

- **String'e Dönüştürme:** Bir sayıyı veya boolean değeri metne çevirmek için **String()** kullanılır.

```
let sayi = 35;
let metinHali = String(sayi);
console.log(typeof metinHali);
```

- **Sayıya Dönüştürme:** Metin içindeki sayıları matematiksel işleme sokabilmek için **Number()** kullanılır.

```
let fiyat = "150";
let gercekSayi = Number(fiyat);
console.log(gercekSayi + 50); /
```

Tür Dönüşümleri –
Otomatik Dönüşüm (Type Coercion)

Kapalı/Otomatik Dönüşüm
(Implicit Conversion)

- **Toplama İşlemi (+):** Eğer taraflardan biri String ise, JS diğerini de metne çevirip yan yana yapıştırır.
  - 5 + "5" ➔ "55" (Metin!)
- **Çıkarma, Çarpma, Bölme (- , * , /):** JS burada "Metinle matematik olmaz, ben bunu sayıya çevireyim" der.
  - "10" - 2 ➔ 8 (Sayı!)
- JavaScript'te program kodlarının akışında değişkenin veri türü, içerisine atanan değere göre örtülü olarak (*otomatik olarak*) değişebilir.
- *Kodlamada dikkat edilmelidir.!!!*

Tür Dönüşümleri – NaN (Not a Number)

JavaScript

- Eğer JS'e imkansız bir dönüşüm yaptırmaya çalışırsanız (örneğin "Merhaba" kelimesini sayıya çevirmek), size küser ve **NaN** sonucunu verir.

```
let sonuc = Number("Selam");
console.log(sonuc); // NaN
console.log(typeof sonuc); /
```

```
> let sText = "20";
  console.log(Number(sText));
  20                                  VM281:2
```

```
> let s2Text = "2deli0";
  console.log(Number(s2Text));
  NaN                                 VM298:2
```

Tür Dönüşümleri - Mantıksal Dönüşüm (Boolean Conversion)

JavaScript

- Bir değeri true veya false yapma işlemidir.
- JS'de her şeyin bir "doğruluk" değeri vardır.

```
> console.log(Boolean("merhaba"));
true                                VM392:1
```

!

```
> console.log(Boolean(" "));
true                                VM406:1
```

```
> console.log(Boolean(""));
false                               VM413:1
```

- **Falsey (Yalancı) Değerler:** 0, "" (boş string), null, undefined, NaN. Bunlar Boolean() içine girince her zaman false çıkar.
- **Truthy (Gerçekçi) Değerler:** Geri kalan her şey (dolu metinler, 0 hariç sayılar, objeler) true çıkar.

```
console.log(Boolean(1));      // true
console.log(Boolean(0));      // false
console.log(Boolean(""));     // false
console.log(Boolean("JS"));   // true
```

Boolean Tür Dönüşümü

- Her zaman FALSE dönen değerler:
  - 0,
  - "" (boş string),
  - null,
  - undefined,
  - NaN.
- Geri kalan her şey (dolu stringler, sayılar) true kabul edilir.

KOD: js-react-ders-sunum-kodlari >> veri_tipleri_donusum

Haydi Cevapla – Soru1

Kodun çıktısı nedir ve neden bu çıktı oluşur?

```
<script>
  let x = "5";
  let y = 5;
  let z = x + y;

  console.log(typeof z);
  console.log(z);
  // Soru: Çıktı nedir ve neden?
</script>
```

Sorular - Verilen kod parçalarını çıktıları nedir?

JavaScript

```
1. console.log(true + 1);
2. console.log("" + 0);
3. console.log( Number(12.5) );
4. console.log( Boolean("False"));
5. console.log( Boolean(true));
```

Tür Dönüşüm fonksiyonları

- **parseInt():**
  - Metin (String) içindeki bir ifadeyi tarar ve bulduğu ilk **tam sayıyı** (Integer) çekip çıkarır. Eğer metin bir sayı ile başlıyorsa, arkasındaki yazıları görmezden gelir.
  - **Kural:** Metin mutlaka bir rakamla başlamalıdır. Harfle başlarsa küser ve NaN döner.

```
let veri1 = parseInt("40");       // Sonuç: 40
let veri2 = parseInt("40.99");    // Sonuç: 40
let veri3 = parseInt("100px");    // Sonuç: 100
let veri4 = parseInt("Hata 50"); // Sonuç: NaN
```

- **parseFloat():**
  - Metin içindeki **ondalıklı sayıları** (Float) olduğu gibi koruyarak çekip alır. Noktadan sonrasını çöpe atmaz.
  - **Kullanım Yeri:** Fiyat hesaplamaları, koordinatlar veya hassas ölçümlerde vazgeçilmezdir.

```
let fiyat = parseFloat("19.99 TL"); // Sonuç: 19.99
let derece = parseFloat("36.6 derece"); // Sonuç: 36.6
let tamSayi = parseFloat("50"); // Sonuç: 50 (Nokta yok
```

KOD: js-react-ders-sunum-kodlari >> veri_tipleri_donusum

Tür Dönüşüm fonksiyonları

- **.toString():**
  - Elinizdeki bir sayı veya boolean değeri metne (String) dönüştürmek için kullanılır.
  - . İle method olarak çağırılır.
  - **Özelliği:** Sayısal bir değerle metni birleştirmeden önce "garantiye almak" için kullanılır.

```
let sayi = 123;
let metin = sayi.toString();
console.log(typeof metin); // "string"

let cevap = true;
console.log(cevap.toString()); // "true"
```

KOD: js-react-ders-sunum-kodlari >> veri_tipleri_donusum

Number() vs parseInt()

- Number("100px") ➔ **NaN** döner (Çünkü içinde 'px' var, "bu sayı değil" der).
- parseInt("100px") ➔ **100** döner (Çünkü içinden sayıyı "ayıklamayı" bilir).

Veri temizse **Number(),**

veri kirliyse (yanında birim vs. varsa) **parseInt()** kullanılır.

KOD: js-react-ders-sunum-kodlari >> veri_tipleri_donusum

JavaScript

Uygulama

Kullanıcıdan doğum yılı bilgisini alan ve yaşını hesaplayan uygulama hazırlayınız.

```
// Kullanıcıdan doğum yılını al (string gelir)
let dogumYili = prompt("Doğum yılınızı giriniz:");

// String'i number'a çevir
dogumYili = parseInt(dogumYili);

// Şu anki yılı al
let simdikiYil = new Date().getFullYear();

// Yaşı hesapla
let yas = simdikiYil - dogumYili;

// Sonucu yazdır
console.log("Yaşınız: " + yas);
```

Soru

Elimizde "150.50TL" ve "25.20TL" şeklinde iki string var. Bunları toplayıp ekrana 175.7 yazdıran kodu yazın.

```
let fiyat1 = "150.50TL";
let fiyat2 = "25.20TL";

// TL kısmını silip sayıya çeviriyoruz
let sayi1 = parseFloat(fiyat1);
let sayi2 = parseFloat(fiyat2);

let toplam = sayi1 + sayi2;

console.log(toplam); // 175.7
```

Operatörler

JavaScript

- Değişkenler ve değerler üzerinde işlem yapmamızı sağlayan özel sembollerdir.
- **Görevleri:** Sayıları toplamak, metinleri birleştirmek veya iki değeri birbiriyle kıyaslamak ve birçok süreç için kullanılırlar.

- Aritmetik Operatörler
- Atama Operatörleri
- Karşılaştırma Operatörleri
- Mantıksal Operatörler
- Bitsel (Bitwise) Operatörler
- Özel Operatörler
  - Koşul (Ternary) Operatörü

Operatör Türleri

KOD: js-react-ders-sunum-kodlari
>> operatorler.html

Aritmetik Operatörler	Atama Operatörleri	Karşılaştırma Operatörleri	Mantıksal Operatörler
+ Toplama **-** Çıkarma ***** Çarpma **/** Bölme **%** bölmeden sonra kalan (Örneğin: 12%5 işlemi 2 değerini verir.) **++** Bir artırma **--** Bir azaltma.	= Atama += Topla ve Ata -= Çıkar ve Ata *= Çarp ve Ata	**==** İstenenler eşitse true döndürür **!=** Eşit değil **>** Büyüktür **<** Küçüktür **>=** Büyük Eşit **<=** Küçük Eşit **===** Denk **!==** Denk değil	**&& ve "And"** İki ifade de doğruysa True döndürür onun dışında false döndürür **|| veya - "Or"** Ifadelerden birisi doğruysa True döndürür. Sadece iki ifade de false oldugunda false dödürür. **! Degil - "Not"** Negatifleştirme işlemi doğruysa True döndürür. true ise false, false ise true yapar

Koşul (Ternary) Operatör

JavaScript

koşul ? doğruysa_bu_çalışır : yanlışsa_bu_çalışır;

- Soru işareti varsa karar veriliyor demektir.
- If-else yapısının tek satıra indirilmiş operatör halidir.
- **Görevi:** Bir koşulu kontrol eder; koşul doğruysa bir işlem, yanlışsa başka bir işlem yapar.

```
let saat = 20;
let selam = (saat < 18) ? "İyi Günler" : "İyi Akşamlar";
console.log(selam);
```

KOD: js-react-ders-sunum-kodlari >> operatorler.html

js-react-ders-sunum-kodlari >>
operatorler_toplu_oyun.html

JavaScript

Operatörlerde Öncelik

Operatörler
. [ ] *[En Yüksek Öncelik]*
( ) new
! ~ - + ++ -- typeof void delete
* / %
+ -
<< >> >>>
< <= > >= in instanceof
== != === !==
&

Operatörler
^
|
&&
||
?:
= += -= *= /= %= <<= >>= >>>= &= ^= |=
, *[En Düşük Öncelik]*

document.write()

- JavaScript'te **HTML sayfasına doğrudan içerik yazdırmak** için kullanılan bir metottur.
- Kullanım:
  - Temel Kullanım:

```
document.write("Merhaba Dünya");
```

  - Değişken ile Kullanım:

```
let ad = "Volkan";
document.write("Merhaba " + ad);
```

  - HTML ile Birlikte Kullanım:

```
document.write("<h1>JavaScript Öğreniyorum</h1>");
```

Değişkenler çift tırnak içerisinde yazılmazlar. Sadece görüntülenmek istenenler çift tırnak içerisinde yazılır

Önemli Not !!!

- document.write() **modern projelerde pek kullanılmaz**
  - çünkü sayfa yüklendikten sonra çalışırsa **sayfayı tamamen silip yeniden yazabilir**.
- Gerçek projelerde genelde şu yöntemler kullanılır:
  - console.log()
  - innerHTML
  - textContent

KOD: js-react-ders-sunum-kodlari >> document_write.html

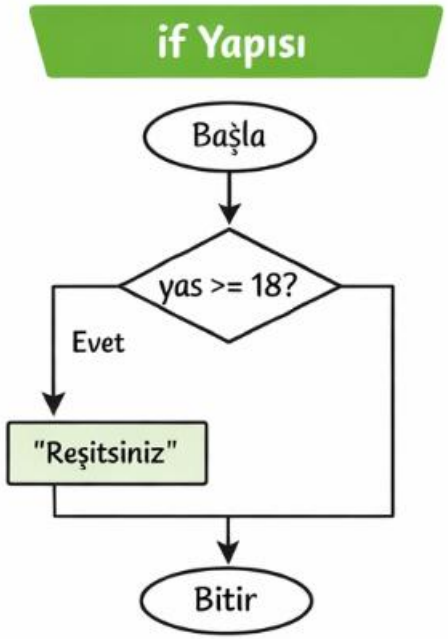
Kontrol İfadeleri

- If
- If-Else
- If … Else – If
- İç İçe If yapıları
- Switch – Case

- Programın akışını belirli şartlara göre yönlendirmemizi sağlar.

if Yapısı (Tek Karar)

```
let yas = 18;

if (yas >= 18) {
    console.log("Reşitsiniz");
}
```



- En temel karar yapısıdır.
- Parantez içindeki koşul **doğru (true)** ise kod bloğu çalışır.
- **Yanlış (false)** ise blok atlanır.

if Yapısı (Tek Karar) - Uygulama

JavaScript

- Kullanıcıdan bir sayı alın ve sayı **pozitif ise** ekrana yazdırın.
- Kullanıcıdan bir sayı alın ve sayı **çift ise** ekrana yazdırın.

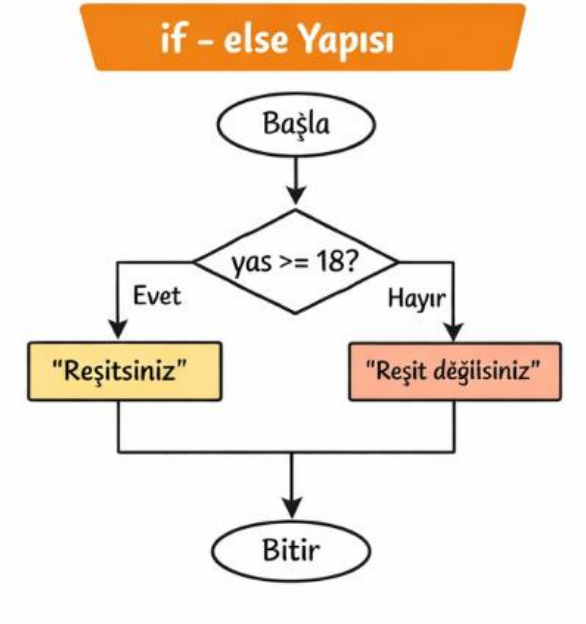
```
<!DOCTYPE html>
<html lang="tr">
<head>
<meta charset="UTF-8">
<title>Tek If Örneği</title>
</head>
<body>

<script>
let sayi = prompt("Lütfen bir sayı
giriniz:");
sayi = Number(sayi);

if (sayi > 0) {
document.write("Girilen sayı pozitif: "
+ sayi);
}

</script>

</body>
</html>
```

KOD: js-react-ders-sunum-kodlari >> if_uygualama.html

if-else Yapısı (İki Karar)

JavaScript

```javascript
let yas = 16;

if (yas >= 18) {
    console.log("Reşitsiniz");
} else {
    console.log("Reşit değilsiniz");
}
```



- Koşul sağlanmadığında (false olduğunda) alternatif bir yol sunar.
- "Ya bu, ya şu" mantığıdır.
- Doğru ise if bloğu, yanlış ise else bloğu çalıştırılır.

if-else Yapısı (İki Karar) - Uygulama

JavaScript

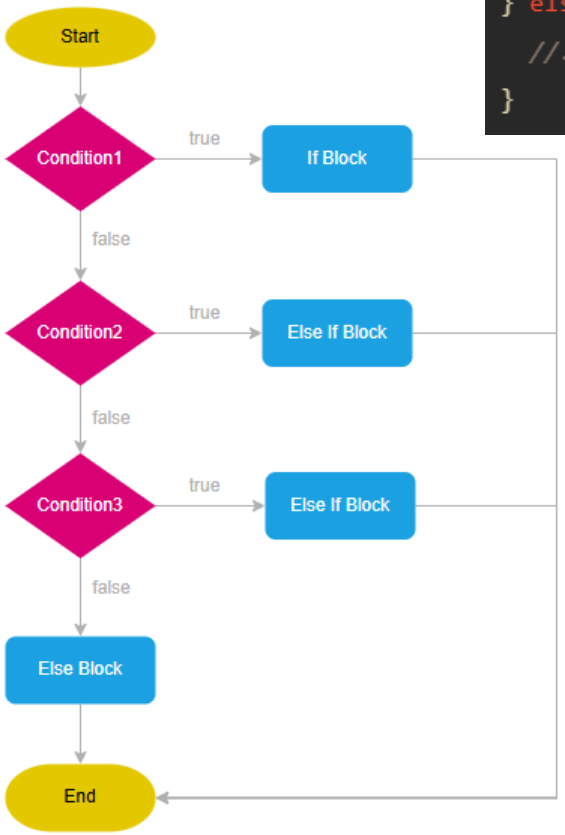
- Kullanıcıdan bir sayı alın ve sayı **çift ise** ekrana “<sayı> çifttir.”, sayı tek sayı ise “<sayı> tektir.” yazdırın.

```
<!DOCTYPE html>
<html lang="tr">
<head>
<meta charset="UTF-8">
<title>Çift/Tek Sayı Kontrolü</title>
</head>
<body>

<script>
    // Kullanıcıdan sayı al
    let sayi = prompt("Lütfen bir sayı giriniz:");
    sayi = Number(sayi);

    // Sayı mı kontrolü
    if (isNaN(sayi)) {
        document.write("Lütfen geçerli bir sayı giriniz.");
    }
    // Çift/tek kontrolü
    else if (sayi % 2 === 0) {
        document.write(sayi + " çifttir.");
    }
    else {
        document.write(sayi + " tektir.");
    }

</script>

</body>
</html>
```

KOD: js-react-ders-sunum-kodlari >> ifelse_uygulama.html

if...else if...else Yapısı (Çoklu Karar)

```
if (condition1) {
  // ...
} else if (condition2) {
  // ...
} else if (condition3) {
  //...
} else {
  //...
}
```



- İkiden fazla koşulun olduğu durumlarda kullanılır.
- JavaScript yukarıdan aşağıya doğru kontrol eder ve **ilk doğru bulduğu** bloğu çalıştırıp çıkar.

if...else if...else Yapısı (Çoklu Karar)

JavaScript

- Kullanıcıdan hava sıcaklığını alan bir JavaScript programı yazınız. Girilen değere göre aşağıdaki mesajları ekrana **document.write()** ile yazdırınız.
  - Sıcaklık **0°C'den küçükse** → *"Hava dondurucu."*
  - Sıcaklık **0 ile 20 arasında ise** → *"Hava serin."*
  - Sıcaklık **20 ile 35 arasında ise** → *"Hava ideal."*
  - Sıcaklık **35°C ve üzerindeyse** → *"Hava çok sıcak."*

```
<script>

let derece = prompt("Lütfen hava sıcaklığını giriniz (°C):");

derece = Number(derece); // string → number dönüşümü

if (derece < 0) {
    document.write("Hava dondurucu.");
}
else if (derece >= 0 && derece < 20) {
    document.write("Hava serin.");
}
else if (derece >= 20 && derece < 35) {
    document.write("Hava ideal.");
}
else {
    document.write("Hava çok sıcak.");
}

</script>
```

- **isNaN(deger):**
  - NaN = Not a Number (Sayı değil)
  - bir değerin **sayı olup olmadığını kontrol eder**.
  - **Dönüş değeri:**
    - **true** → sayı değil
    - **false** → sayı

```
isNaN("abc")   // true
isNaN(25)      // false
isNaN("15")    // false
isNaN("15a")   // true
```

if...else if...else Yapısı (Çoklu Karar)

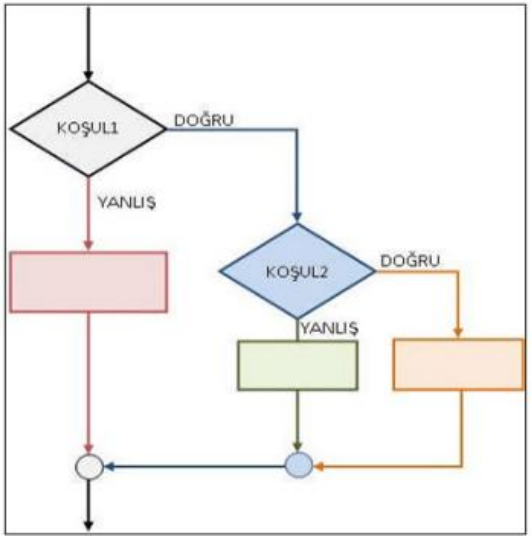
- Kullanıcıdan **sınav notunu** alın ve not aralığına göre mesaj yazdırın:

Not Aralığı	Mesaj
90–100	AA
70–89	BB
50–69	CC
0–49	FF
Geçersiz	Hatalı not

KOD: js-react-ders-sunum-kodlari >> if_elseif_if_uygulama.html

İç İçe if (Nested If) yapısı

- Bir koşulun gerçekleşmesi, başka bir alt koşula bağlıysa kullanılır.




```
if(koşul1)
{
    if(koşul2)
        komut1;
    else
    {
            if(koşul3)
            komut2;
        komut3;
        komut4;
    }
}
else
    komut5;
```

Kullanıcının yaşına göre ehliyet alıp alamayacağını kontrol edelim:

•Eğer yaş **18 ve üzeri** ise:

- Eğer kullanıcı **sürücü kursuna kayıtlıysa** → "Ehliyet alabilirsiniz"
- Değilse → "Önce sürücü kursuna kayıt olun"

•Eğer yaş **18'den küçük** ise → "Ehliyet alamazsınız"

KOD: js-react-ders-sunum-kodlari >> nested_if.html

İç İçe if (Nested If) yapısı - Uygulama

```
let kullaniciAdi = "admin";
let sifre = "1234";

if (kullaniciAdi === "admin") {
    if (sifre === "1234") {
        console.log("Giriş
başarılı.");
    } else {
        console.log("Şifre
hatalı.");
    }
} else {
    console.log("Kullanıcı adı
bulunamadı.");
}
```

Bu kod parçası aşağıdaki girdiler için ne çıktı verir:

1. `kullaniciAdi = "admin";`
   `sifre = "1234";`
2. `kullaniciAdi = “user";`
   `sifre = "1234";`
3. `kullaniciAdi = "admin";`
   `sifre = “3435";`

KOD: js-react-ders-sunum-kodlari >> nested_if_uygulama.html

switch-case Yapısı

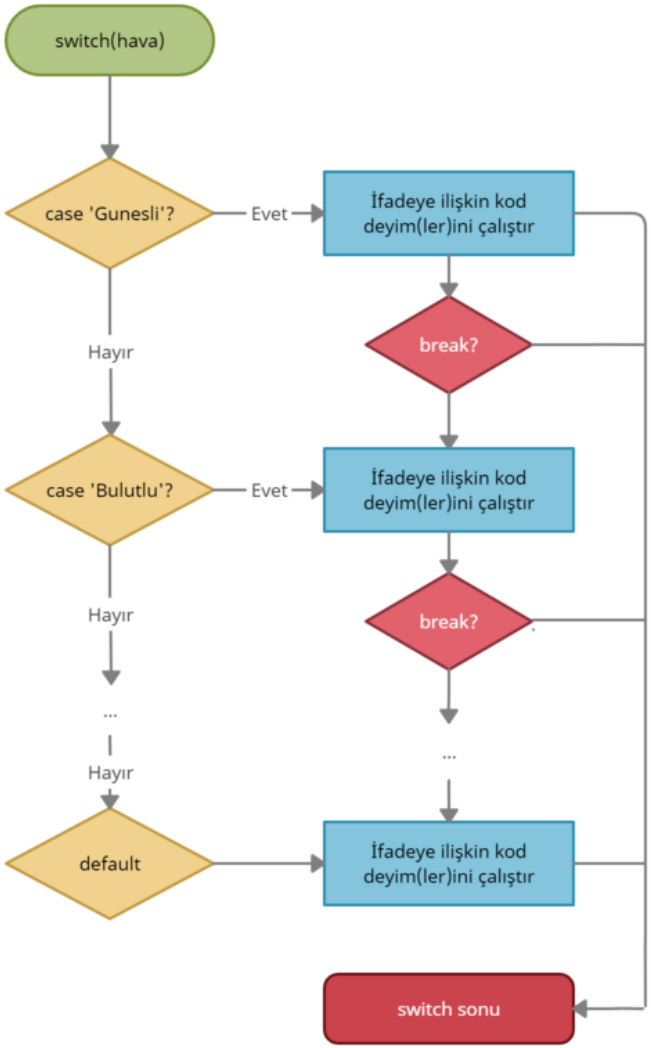
- Bir değişkenin alabileceği **belirli sabit değerler** varsa, if-else kalabalığından kurtulmak için kullanılır.
- Eşitlik kontrolü (strict equality ===) yapar.
- **!!! Break Komutu:** switch içinde break kullanılmazsa, kod "akmaya" devam eder ve eşleşen case'den sonraki tüm kodları çalıştırır.
- **Performans:** Çok fazla else-if yerine, sabit değerler kontrol ediliyorsa switch daha okunaklı ve bazen daha hızlıdır.

Girilen hava durumuna göre çıktı veren uygulama

switch-case Yapısı - Uygulama

Girilen hava durumuna göre çıktı veren uygulama

```
let hava = "Gunesli";
switch(hava) {
  case "Yagmurlu":
      console.log("Semsiyeni yanina almayi
unutma");
      break;
  case "Gunesli":
      console.log("Hafif giyin");
  case "Bulutlu":
      console.log("Disari cik");
      break;
  case "Karlı":
      console.log("Kalin giyin");
      break;
  case "Firtinali":
      console.log("Bir süre disari cikma");
      break;
  default:
      console.log("Bilinmeyen hava durumu:" ,
hava);
}
```



KOD: js-react-ders-sunum-kodlari >> switchcase_uygulama.html

switch-case Yapısı - Gün İsmi Yazdırma

JavaScript

Gün Numarası	Gün
1	Pazartesi
2	Salı
3	Çarşamba
4	Perşembe
5	Cuma
6	Cumartesi
7	Pazar
Diğer	Geçersiz gün

```
<script>
    let gun = prompt("Lütfen gün numarasını giriniz (1-7):");
    gun = Number(gun);

    switch(gun) {
        case 1:
            document.write("Pazartesi");
            break;
        case 2:
            document.write("Salı");
            break;
        case 3:
            document.write("Çarşamba");
            break;
        case 4:
            document.write("Perşembe");
            break;
        case 5:
            document.write("Cuma");
            break;
        case 6:
            document.write("Cumartesi");
            break;
        case 7:
            document.write("Pazar");
            break;
        default:
            document.write("Geçersiz gün");
    }
</script>
```

KOD: js-react-ders-sunum-kodlari
>> switchcase_ornek_soru.html

Kontrol Yapıları Ders Uygulaması

JavaScript

SORU 1: Bir sinema salonu bilet fiyatlarını aşağıdaki kurallara göre hesaplamak istiyor:

- Yaş < 12 ise **çocuk bileti** uygulanır → %50 indirim
- Yaş 12–25 arasında ve öğrenci ise **genç öğrenci bileti** uygulanır → %30 indirim
- Yaş 65+ ise **yaşlı bileti** uygulanır → %40 indirim

Diğer durumlarda **tam bilet** uygulanır

Kullanıcıdan **yaş** ve **öğrenci olup olmadığını (true/false)** alıp **bilet fiyatını hesaplayan** ve ekrana **hangi bilet uygulandığını ve ödenecek tutarı** yazdıran bir JavaScript kodu yazınız.

SORU 2:Kullanıcının **kilo** ve **boy** bilgilerini alarak **Vücut Kitle İndeksini (VKI)** hesaplayan bir program yazınız.

- VKI < 18.5 → Zayıf
- VKI 18.5 – 24.9 → Normal kiloda
- VKI 25 – 29.9 → Fazla kilolu
- VKI >= 30 → Obezite → "Sağlık kuruluşuna danışın" mesajı gösterilsin

Not:

VKI hesaplaması: VKI = kilo / (boy * boy)

Program **nested if** kullanmalıdır

Hesaplanan VKI ve kategori **ekrana yazdırılmalıdır**

Ödev

Bir ATM'de kullanıcı aşağıdaki işlemleri yapabilsin:

- **Bakiye görüntüleme** → işlem kodu "1"
- **Para çekme** → işlem kodu "2" (çekilecek tutar, bakiyeden büyük olamaz)
- **Para yatırma** → işlem kodu "3"

Kullanıcıdan **işlem kodu** alın ve:

1. Eğer "1" seçilmişse bakiyeyi ekrana yazdırın
2. Eğer "2" seçilmişse çekilecek tutar bakiyeden fazla ise "Yetersiz bakiye!", değilse işlemi yapıp kalan bakiyeyi yazdırın
3. Eğer "3" seçilmişse yatırılacak tutarı bakiyeye ekleyip yeni bakiyeyi yazdırın
4. Geçersiz işlem kodu girilirse "Geçersiz işlem kodu seçtiniz." mesajı gösterin

Not: İşlemler için switch-case ve gerekirse if kullanınız.

JavaScript

Hata Yönetimi / try-catch

- Hata yönetimi (exception handling), kodunuzun çalışma esnasında beklenmedik bir durumla karşılaştığında çökmesini engellemek ve bu hataları zarif bir şekilde yönetmek için kullanılır.

- Temel Hata Yönetimi Yapıları
  - try...catch Bloğu
  - finally (Sonunda)
  - throw (Fırlat)

Komut	Türkçe Karşılığı	Görevi
`try`	Dene	Riskli kodu çalıştırır.
`catch`	Yakala	Hata olduğunda devreye girer.
`throw`	Fırlat	Özel bir hata mesajı oluşturur.
`finally`	Sonunda	Her koşulda en son çalışır.

try...catch.. (finally) Bloğu

- Hata oluşabilecek kodların denetlendiği temel yapıdır.
- **try...catch** ifadesi, denenmek üzere bir veya daha fazla ifadeden oluşan bir kod bloğunu işaretler ve bir istisna (exception) oluştuğunda verilecek bir veya daha fazla yanıtı belirtir.
- Eğer bir istisna oluşursa, **try...catch** ifadesi bu hatayı yakalar.

try bloğunun başarılı olması beklenir; ancak başarılı olmazsa kontrolün **catch bloğuna geçmesi** isternir

- **try (Dene):**
  - Hata oluşma ihtimali olan kodların yazıldığı bloktur.
  - Bir veya daha fazla ifadenin bulunduğu ve çalıştırılmasının denendiği kısımdır.
- **catch (Yakala):**
  - try bloğu içinde bir istisna (exception) oluşursa ne yapılacağını belirleyen ifadeleri içerir.
  - Eğer try bloğu içinde bir hata oluşursa, program akışı anında buraya geçer. Hata nesnesini (error) parametre olarak alır.
- **finally (Sonunda):**
  - Hata oluşsa da oluşmasa da **mutlaka** çalışması gereken kodlar buraya yazılır.
  - Genellikle dosya kapatma veya bağlantı kesme gibi temizlik işlemleri için kullanılır.

try...catch.. (finally)

- Eğer try bloğu içindeki herhangi bir ifade (ya da try bloğu içinden çağrılan bir fonksiyon) bir istisna oluşturursa, kontrol **hemen catch bloğuna aktarılır**.
- Eğer try bloğunda hiçbir istisna oluşmazsa, **catch bloğu atlanır**.
- **finally** bloğu ise try ve catch blokları çalıştıktan sonra, fakat **try...catch ifadesinden sonra gelen kodlardan önce** çalıştırılır.

KOD: js-react-ders-sunum-kodlari >> try_catch_finally.html

JS Kontrol Yapıları – Soru 1

1. Aşağıdaki kod bloğu çalıştırıldığında konsol çıktısı ne olur?

```
let x = 10;
if (x > 5) {
if (x < 15) {
console.log('A');
}
console.log('B');
}
```

A. A ve B

B. Sadece A

C. Sadece B

D. Hiçbir şey yazdırılmaz

JS Kontrol Yapıları – Soru 2

2. Bir switch-case yapısında, hiçbir case eşleşmediğinde çalışacak olan kod bloğu hangisi ile tanımlanır?

A. break

B. else

C. fallback

D. default

JS Kontrol Yapıları – Soru 3

3. Aşağıdaki if-else yapısındaki boşluğu, 'puan' 50'den küçükse 'Kaldı', değilse 'Geçti' yazdıracak şekilde doldurunuz:

```
if (puan < 50) {
console.log('Kaldı');
} _____ {
console.log('Geçti');
}
```

A. then

B. case

C. else if

D. else

JS Kontrol Yapıları – Soru 4

4. Aşağıdaki kodun çıktısı nedir?

```
let meyve = 'Elma';
switch(meyve) {
case 'Armut': console.log('1'); break;
case 'Elma': console.log('2');
case 'Muz': console.log('3'); break;
default: console.log('4');
}
```

A. 2 ve 3

B. Hata verir

C. 2

D. 2, 3 ve 4

JS Kontrol Yapıları – Soru 5

5. Kullanıcının girdiği sayının hem pozitif hem de çift olduğunu kontrol etmek için en uygun 'İç İçe If' yapısı hangisidir?

A. if (sayi > 0 || sayi % 2 === 0) { ... }

B. if (sayi > 0) { if (sayi % 2 === 0) { ... } }

C. if (sayi > 0) else if (sayi % 2 === 0) { ... }

D. if (sayi % 2 === 0) { else { ... } }

JS Kontrol Yapıları – Soru 6

6. Aşağıdaki kodda 'merhaba' yazdırılması için 'yas' değişkeni ne olmalıdır?

```
if (yas > 18) {
if (yas < 25) {
console.log('merhaba');
}
}
```

A. 18

B. 30

C. 20

D. 25

JS Kontrol Yapıları – Soru 7

7. JavaScript'te 'Strict Equality' (===) kullanan kontrol yapısı hangisidir?

A. Hepsi

B. if-else

C. if

D. switch

JS Kontrol Yapıları – Soru 8

8. Aşağıdaki kodun çıktısı nedir?

```
let a = 5, b = 10;
if (a > b) {
console.log('A Büyük');
} else if (a === b) {
console.log('Eşit');
} else {
console.log('B Büyük');
}
```

A. A Büyük

B. B Büyük

C. Hata

D. Eşit

JS Kontrol Yapıları – Soru 9

9. Switch-case yapısında 'break' komutunun görevi nedir?

A. Programı tamamen durdurur

B. Default bloğuna yönlendirir

C. Bir sonraki case'e geçmeyi engeller

D. Hata mesajı verir

JS Kontrol Yapıları – Soru 10

10. Aşağıdaki boşluğu hangi operatör ile doldurursak hem sayı 10'dan büyükse hem de 'aktif' değişkeni true ise kod çalışır?

```
if (sayi > 10 ____ aktif === true)
```

A. &&

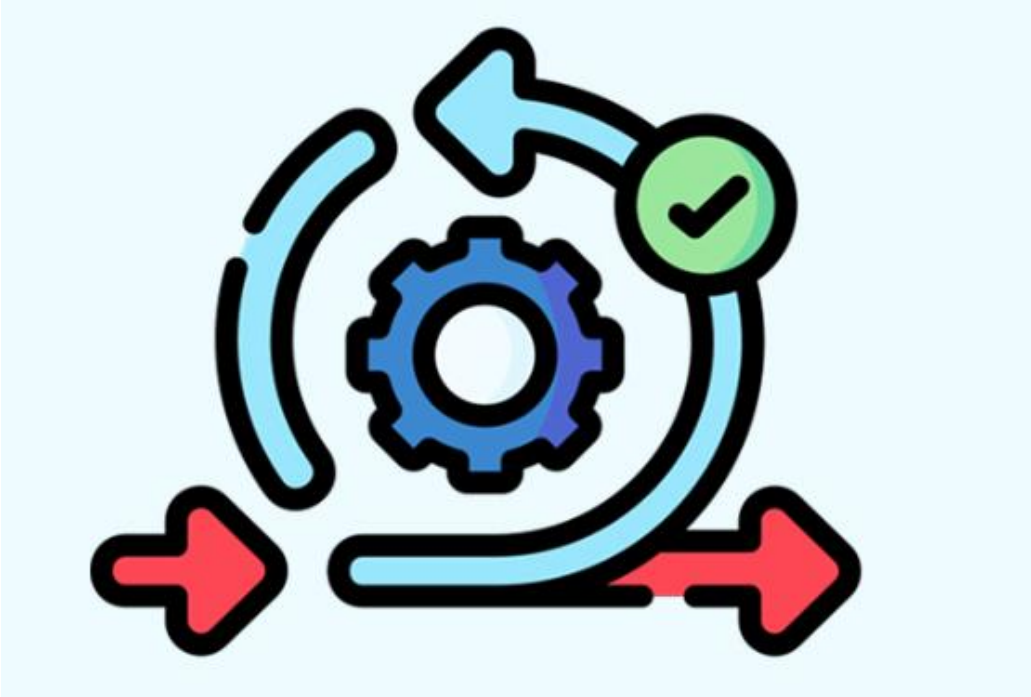
B. !

C. ==

D. ||

Döngüler

Belirli bir kod bloğunu tekrar tekrar çalıştırmak istediğimizde kullandığımız yapılarıdır.



Yazılım ve Uygulama Geliştirici olarak; Tekrarlayan İşlemleri Otomatikleştirin

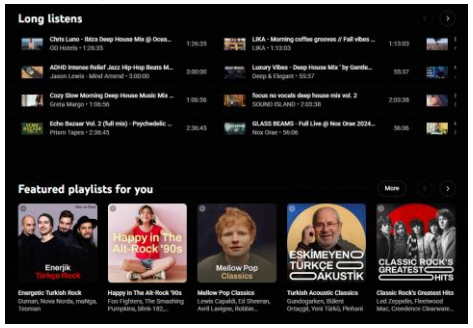
Bir işlemi 100 kez yapmanız gerekse, tek tek mi yazarsınız yoksa bir yolunu mu bulursunuz?

Amacı: Kodun daha temiz, kısa ve sürdürülebilir olmasını sağlamaktır.

Görevi: Bir kod bloğunu belirli bir sayıda üst üste çalıştırmaktır.

Gerçek Dünyadan – Sistem geliştirirken

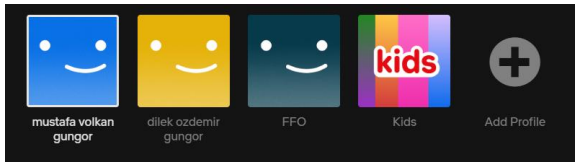
Spotify/YouTube Çalma Listeleri



Spotify, sizin için binlerce şarkılık bir çalma listesini ekrana yazdırırken bir döngü kullanır. Her şarkı, listedeki bir öğedir ve döngü, her birini sırayla gösterir.

Instagram Akışınız

Instagram, sizin takip ettiğiniz kişilerin gönderilerini akışınızda sıralarken for döngülerinden faydalanır. Her bir gönderi, akışta gösterilecek bir sonraki öğedir.

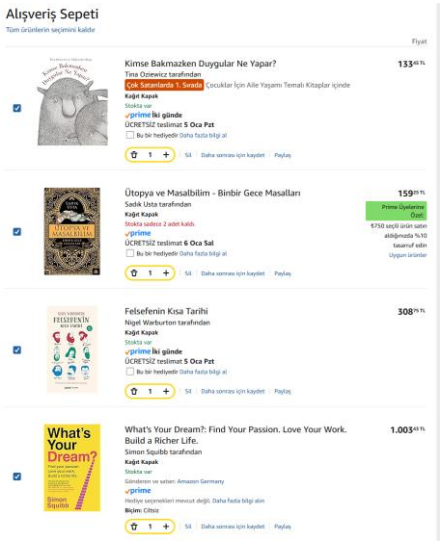


Netflix Film Önerileri

Netflix, izleme geçmişinize göre size 50 film önerisi sunarken bir döngü çalıştırır. Her film önerisi, döngüdeki bir öğedir.

Amazon Sepetiniz

Amazon'da sepetinizdeki ürünleri listelerken, sistem bir döngü kullanarak her bir ürünü sırayla gösterir ve toplam tutarı hesaplar.



For Döngüsü

Başlangıç → Koşul → İlerleme

Bir kod blokunu belirli bir sayıda ve üst üste çalıştırmak için kullanılan bir döngüdür.

Nereden Başlayacağını Bilmek (**Başlangıç**)

Motosikletçi başlangıç çizgisinden start alıyor. Döngü de ilk adımını.

Ne Zaman Duracağını Bilmek (**Koşul**)

"Tur sayısı 20'den küçük mü / eşit mi?" Döngü koşul karşılandığında devam eder

FALSE

TRUE

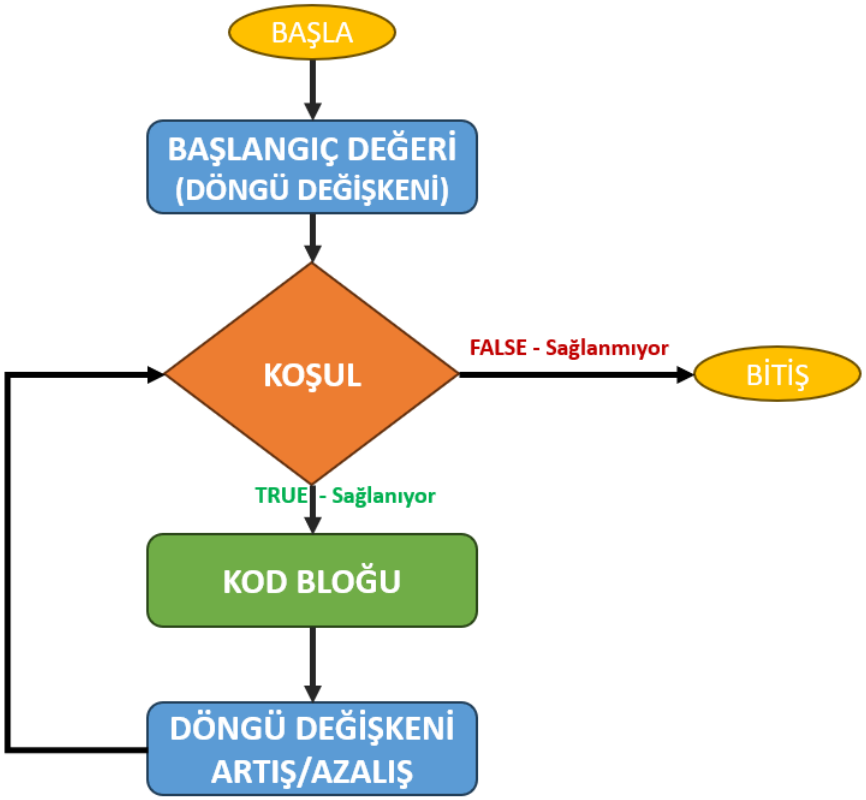
Her Turda Ne Kadar İlerleyeceğini Bilmek (**İlerleme** /Artış)

Her tur sonunda tur sayısı 1 artırılır, döngü de her tekrarda güncellenir.

For Döngüsünün İşleyişi

```
for (başlangıç değeri; koşul; artış/azalış) {
  // tekrarlanacak kod bloğu
}
```

i	Koşul Durumu	Çıktı	i++
1	1 <= 5 true	Tur 1	2
2	2 <= 5 true	Tur 2	3
3	3 <= 5 true	Tur 3	4
4	4 <= 5 true	Tur 4	5
5	5 <= 5 true	Tur 5	6
6	6 <= 5 false	ÇIKIŞ	




```
for (let i = 1; i <= 5; i++) {
 console.log("Tur " + i);
}
```

```
for (let i = 5; i >= 1; i--) {
 console.log("Tur " + i);
}
```

1

Başlangıç Değeri

Döngü başladığında sadece bir kez çalıştırılır. Genellikle burada döngü değişkeni tanımlanır ve bir başlangıç değeri atanır.

2

Koşul (Condition)

Her yinelemeden önce değerlendirilen bir mantıksal ifadedir. Koşul true ise döngü devam eder, false ise döngü sonlanır.

3

Yineleyici (Iterator)

Her yinelemeden (**döngü gövdesindeki işlemler bittikten**) **sonra çalıştırılır**. **Döngü değişkenini günceller** (i++; i = i + 2; i--).

Klasik For Döngüsü

- **Pizza Dilimleme:** 8 dilimli bir pizzayı paylaşmak
- **Sayıların Toplamı:** Belirli bir aralıktaki sayıları toplamak (örn: i++ ile 1'er, i+2 ile 2'şer artış)
- **Çarpım Tablosu:** 5ler için çarpım tablosunu oluşturmak

```
for (let dilim = 1; dilim <= 8; dilim++) {
   console.log(dilim + ". dilimi aldım!");
}
```

```
let toplam = 0;
for (let i = 1; i <= 100; i++) {
            toplam += i;
            console.log(i);
}
console.log("1'den 100'e kadar olan sayıların toplamı:", toplam);
```

```
for (let i = 1; i <= 10; i++) {
console.log("5 x " + i + " = " + (5 * i));}
```

Output:

```
1. dilimi aldım!
2. dilimi aldım!
3. dilimi aldım!
4. dilimi aldım!
5. dilimi aldım!
6. dilimi aldım!
7. dilimi aldım!
8. dilimi aldım!
```

Output:

```
1
2
3
97
98
99
100
1'den 100'e kadar olan sayıların toplamı: 5050
```

KOD: js-react-ders-sunum-kodlari
>> dongu_for_ornekleri.html

Soru -1

1. Aşağıdaki JavaScript kod bloğu çalıştırıldığında konsola en son hangi sayı yazdırılır?

JavaScript

```javascript
for (let i = 2; i <= 10; i += 3) {
  console.log(i);
}
```

A. 10

B. 7

C. 8

D. 11

Sınıf Sorusu-1

- JavaScript programlama dilinde **Math.random()** fonksiyonunu kullanarak **1 ile 20 arasındaki sayılar için işlem yapan bir program** yazınız. Programda **for döngüsü** ve **if koşul yapıları** kullanılmalıdır.

Program aşağıdaki kurallara göre çalışmalıdır:

- Program **1'den 20'ye kadar olan sayıları** bir for döngüsü ile kontrol etmelidir.
- Her sayı için **1 ile 10 arasında rastgele bir sayı** üretiniz.
- Aşağıdaki koşullara göre ekrana çıktı veriniz:
  - Eğer sayı **tek ise**, sayının **rastgele üretilen sayı ile çarpımını** ekrana yazdırınız.
  - Eğer sayı **çift ise**, sayının **karesini** ekrana yazdırınız.
  - Eğer sayı **10'a tam bölünüyorsa**, "Merhaba ...." şeklinde bir mesaj yazdırınız.
    - Noktalı yere sayının değeri **string olarak** yazdırılmalıdır.
- Çıktılar **document.write()** kullanılarak ekrana yazdırılmalıdır.

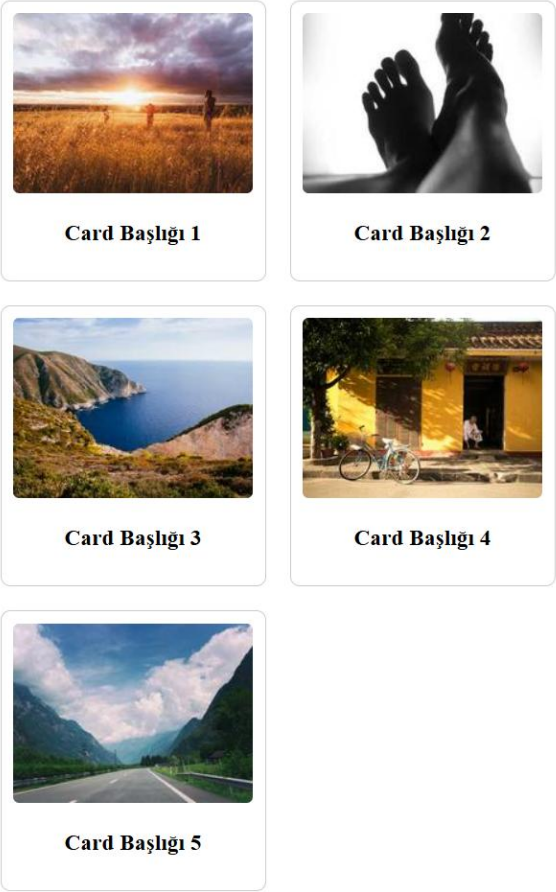
Sınıf Sorusu-1

- Yandaki *Math.random()* fonksiyonunu kullanarak, 1 ile 20 arasındaki sayıları aşağıdaki koşula göre ekrana '*document.write*' kullanarak yazdıran bir program yazınız.
  - Eğer sayı tek ise, sayı ile rastgele bir sayını çarpımını yazdırsın
  - Sayı çift ise karesini yazdırsın.
  - Sayı 10'a tam bölünüyorsa 'Merhaba .....' boşluk olan yere sayının değerini string olarak yazıdırsın.

- Math.random() fonksiyonu 0(dahil) ile 1 (hariç) arasında değer döndür.
- Math.floor(Math.random() * 10); [0,9] dahil olmak üzere aralarındaki tamsayı değerleri döndür.

KOD: js-react-ders-sunum-kodlari >> dongu_for_sinif_sorusu_1.html

For ve HTML DOM işlemleri



JavaScript kullanarak **sayfaya dinamik olarak 5 adet card oluşturan bir program yazınız.**

Koşullar:

- Programda **for döngüsü kullanılacaktır.**
- Her card bir **div** elemanı olacaktır.
- Card içinde:
  - Bir **resim (img) –** 'https://picsum.photos/200/150?random=' sitesinden alınabilir
  - Bir **başlık (h3)** bulunacaktır.
- Başlıklar **"Card Başlığı 1", "Card Başlığı 2" ...** şeklinde olmalıdır.
- Card elemanları **document.body.appendChild()** ile sayfaya eklenmelidir.

KOD: js-react-ders-sunum-kodlari >> dongu_for_dom_kullanim.html

For ve HTML DOM işlemleri

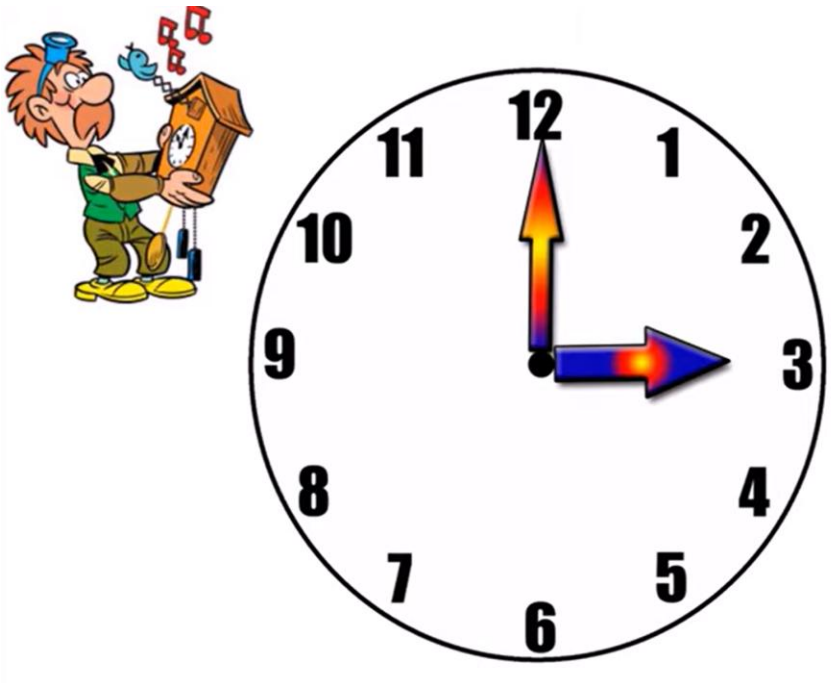
- Random fotoğraf üretmek yerine dosyalar proje içinde olunca bu uygulamayı nasıl yapabiliriz?

KOD: js-react-ders-sunum-kodlari >>dongu_for_dom_uygulama >> dongu_for_dom_uygulama.html

İç içe for döngüsü (nested loops)

JavaScript

- Bir döngünün gövdesi içinde başka bir döngünün yer almasıdır.
- Genellikle **tablolar**, **matrisler** (satır-sütun yapıları) veya bir listenin her elemanını başka bir listenin tüm elemanlarıyla kıyaslamak gerektiğinde



bir saatin yelkovan ve akrep ilişkisi gibi düşünebilirsiniz

İç içe for döngüsü (nested loops)

JavaScript

```
<script>
  for (let i = 1; i <= 3; i++) {
    // Dış döngü (Satırlar)
    document.write("Dış döngü adım: " + i + "<br>");

    for (let j = 1; j <= 2; j++) {
      // İç döngü (Sütunlar)
      document.write("  İç döngü adım: " + j + "<br>");
    }
  }
</script>
```

Dış döngü adım: 1
İç döngü adım: 1
İç döngü adım: 2
Dış döngü adım: 2
İç döngü adım: 1
İç döngü adım: 2
Dış döngü adım: 3
İç döngü adım: 1
İç döngü adım: 2

- SORU: 5'lere kadar olan çarpım tablosunu yazdırınız.

Beklenen Çıktı

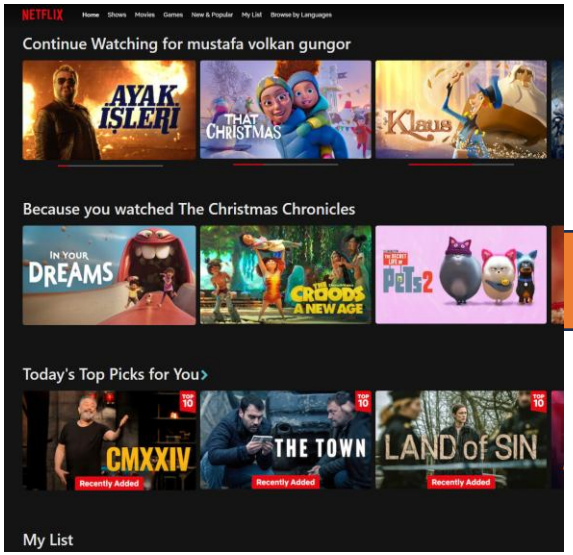
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5

2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10

KOD:
- js-react-ders-sunum-kodlari >> dongu_icice_for.html
- js-react-ders-sunum-kodlari >> dongu_carpim_tablosu.html

KOD: js-react-ders-sunum-kodlari >>dongu_for_dom_uygulama >> dongu_icicefor_dom_uygulama.html

İç İçe For



İç içe döngüde, dış döngü bir adım attığında, iç döngü TAMAMEN çalışır ve biter. Sonra dış döngü bir sonraki adıma geçer ve iç döngü TEKRAR baştan başlar.

- Çarpım Tablosu!
- Bir sinema salonunda 5 sıra var. Her sırada 8 koltuk var. Tüm koltukları numaralandıracağız!
- Bir görüntü 10x10 pixel. Her pikseli koordinatlarıyla numaralandıralım!
- Excel'deki tablo hücreleri, her hücreye X yazdır

```
BAŞLA
container değişkenini sayfadaki container elemanına ata
sayac = 1
FOR i = 1 → 3
yeni bir row oluştur
FOR j = 1 → 3
yeni bir card oluştur
yeni bir img elemanı oluştur
img kaynağını "images/image" + sayac + ".jpg" olarak ayarla
yeni bir başlık oluştur
başlık içeriği = "Card " + sayac
img elemanını card içine ekle
başlığı card içine ekle
card elemanını row içine ekle
sayac = sayac + 1
END FOR
row elemanını container içine ekle
END FOR
BİTİR
```

KOD: js-react-ders-sunum-kodlari >> dongu_for_SONSUZ-DIKKAT.html

JavaScript

Sonsuz Döngü

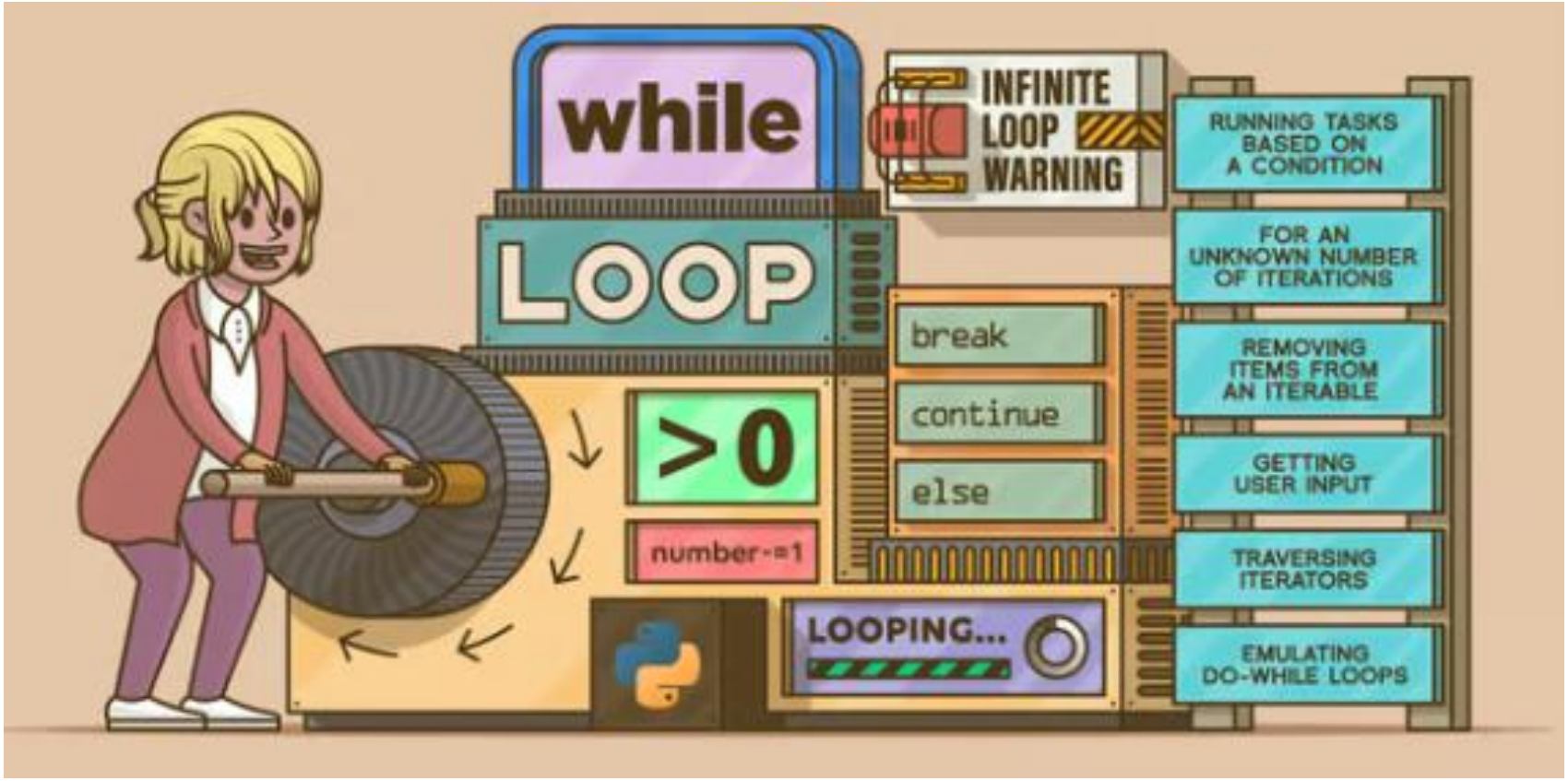
#	HATA TİPİ	❌ YANLIŞ KOD	✅ DOĞRU KOD	NEDEN SONSUZ?
1	Artış Unutulmuş	`for(let i=0; i<10; )`	`for(let i=0; i<10; i++)`	i hiç artmıyor, hep 0 kalıyor
2	Yanlış Yön (Artış)	`for(let i=10; i>0; i++)`	`for(let i=10; i>0; i--)`	Geriye gitmeli ama ileriye gidiyor
3	Yanlış Yön (Azalış)	`for(let i=0; i<100; i--)`	`for(let i=0; i<100; i++)`	İleriye gitmeli ama geriye gidiyor
4	Hep True Koşul	`for(let i=0; i>=0; i++)`	`for(let i=0; i<10; i++)`	Pozitif sayı her zaman ≥0
5	True Koşulu	`for(let i=0; true; i++)`	`for(let i=0; i<10; i++)`	Koşul her zaman doğru
6	Atlanan Değer	`for(let i=0; i!=10; i+=2)`	`for(let i=0; i<10; i+=2)`	0,2,4,6,8,12... (10'u atlar)
7	Ondalık Hata	`for(let i=0; i!=1; i+=0.1)`	`for(let i=0; i<1; i+=0.1)`	Ondalık kesin değil, tam 1 olmaz
8	Yanlış Değişken	`for(let i=0; i<10; j++)`	`for(let i=0; i<10; i++)`	j artıyor ama i kontrol ediliyor
9	Sıfır Artış	`for(let i=0; i<10; i+=0)`	`for(let i=0; i<10; i++)`	0+0+0=0, hiç artmıyor
10	İçerde Sıfırlama	`for(let i=0; i<10; i++) { i=0; }`	`for(let i=0; i<10; i++)`	Her seferinde başa dönüyor

Bilgisayarınız dondu mu hiç?

Tarayıcı yanıt vermiyor mu?

Chrome 'Sayfa yanıt vermiyor' dedi mi?

While döngüsü



While Döngüsü

- Bir koşul **doğru (true)** olduğu sürece çalışmaya devam eder.
- Döngü sayısının belirsiz olduğu, sadece bir şarta bağlı olduğu durumlarda idealdir.

```
// Döngü değişkeni tanımlama ve değer atama
while (şart)
{
    // while döngüsü kapsamı
    // Değişkenin değerini arttırma/azaltma
}
```

While örnek

Kullanıcı 0 girene kadar sayılar girecek ve program girilen tüm pozitif sayıların toplamını hesaplayacak.

KOD: js-react-ders-sunum-kodlari >> dongu_while_uygulama-2.html

JavaScript

```
BAŞLA
toplam = 0
sayi = -1   // Döngünün başlaması için başlangıç değeri 0'dan farklı
WHILE sayi ≠ 0
   sayi = kullanıcıdan sayı al (prompt)
   sayi = sayıyı tam sayıya çevir (parseInt)

   IF sayi > 0 THEN
      toplam = toplam + sayi
   ELSE IF sayi < 0 THEN
      kullanıcıya uyarı ver ("Negatif sayı giremezsiniz!")
   END IF
END WHILE
sonucu ekrana yazdır: "Girilen pozitif sayıların toplamı: " + toplam
BİTİR
```

While - Sonsuz Döngü Nasıl Oluşur?

- **Yanlış Koşul:** Koşul her zaman doğrudur.
- **Değişkeni Güncellemeyi Unutmak:** Koşulu kontrol eden değişken döngü içinde hiç değişmiyordur.

dongu_while_sonsuz_dongu.html

Dikkat!! While

JavaScript

- **while** anahtarına iliştirilen parantezler içerisine yazılan şart, genelde döngüye giremeden oluşturulmuş bir değişken ile kurgulanır.
- Bunun yanında döngü şartında kullanılan değişkenin değeri, döngü içerisinde işlemler yapıldıktan sonra azaltılmalı ya da arttırılmalıdır. Aksi durumda sonsuz bir döngü kurgulanmış olur.

```
let sayac = 1;
// 1. Başlangıç değeri (Döngü değişkeni)

while (sayac <= 5) {
  // 2. Şart: sayac 5'ten küçük veya eşit olduğu sürece çalış
  document.write("Sayı: " + sayac + "<br>");

  sayac++;
  // 3. Artırma: Eğer bunu unutursanız "Sonsuz Döngü" oluşur!
}

document.write("Döngü bitti. Sayac: " + sayac);
```

1'den 5'e kadar sayıları ekrana yazdıran basit bir JavaScript örneği

KOD: js-react-ders-sunum-kodlari >> dongu_while_ornek_1.html

While Uygulama

JavaScript kullanarak bir uygulama yazınız. Uygulama aşağıdaki işlemleri gerçekleştirmelidir:

- Kullanıcıdan **doğum yılı** bilgisi alınacaktır.
- Kullanıcı **4 haneli ve geçerli bir yıl** girene kadar uygulama tekrar çalışacaktır.
  - 1000–güncel yıl arası değer geçerli kabul edilecektir.
  - Girilen değer sayı ile başlar ise girilen değerin ilk 4 hanesi alınabilir.
  - Geçersiz değer girildiğinde kullanıcıya uyarı verilecektir.
- Kullanıcı geçerli bir yıl girdiğinde program:
  - Yaşı hesaplayacak (yaş = mevcut yıl - doğum yılı)
  - Yaşa göre aşağıdaki kategorileri belirleyecektir:
    - 0–12 → "Çocuk"
    - 13–19 → "Genç"
    - 20–59 → "Yetişkin"
    - 60+ → "Yaşlı"
- Hesaplanan yaş ve kategori **document.write veya alert ile kullanıcıya gösterilecektir**.

KOD: js-react-ders-sunum-kodlari >> dongu_while_uygulama.html

Do...while döngüsü

Do...while döngüsü

- do...while döngüsü, koşul ne olursa olsun kod bloğunu en az bir kez çalıştırır. (while döngüsünden dan farkı)

```
do {
  // Çalıştırılacak kodlar
  console.log("Bu satır en az bir kez görünecek.");
} while (koşul);
```

Do...while örnek

- Pozitif sayı girene kadar tekrar sor.

```
<script>
  let sayi;

  do {
    sayi = prompt("Pozitif bir sayı giriniz:");
    sayi = parseInt(sayi);

    if (isNaN(sayi) || sayi <= 0) {
      alert("Lütfen pozitif bir sayı giriniz!");
    }
  } while (isNaN(sayi) || sayi <= 0);

  alert("Girdiğiniz pozitif sayı: " + sayi);
</script>
```

akış

1. **do bloğu** → kullanıcıya en az bir kez prompt gösterilir.
2. Kullanıcı **geçersiz değer** girerse alert ile uyarılır.
3. Koşul (isNaN(sayi) || sayi <= 0) sağlandığı sürece döngü devam eder.
4. Kullanıcı geçerli bir sayı girince döngü biter ve sayı ekrana yazdırılır.

KOD: js-react-ders-sunum-kodlari >> dongu_doWhile.html

do...while uygulama

JavaScript kullanarak bir program yazınız. Program aşağıdaki işlemleri gerçekleştirmelidir:

- Kullanıcıdan **0–100 arası bir not** alınız.
- Kullanıcı geçersiz bir değer girerse, program **tekrar soracak** (0–100 dışındaki sayılar veya sayısal olmayan değerler geçersizdir).
- Geçerli bir sayı girildiğinde program, kontrol yapısı kullanarak girilen notu harf notuna dönüştürecektir:
  - 90–100 → "AA"
  - 80–89 → "BA"
  - 70–79 → "BB"
  - 60–69 → "CB"
  - 50–59 → "CC"
  - 0–49 → "FF"
- Sonuç **document.write** veya **alert** ile kullanıcıya gösterilecektir.

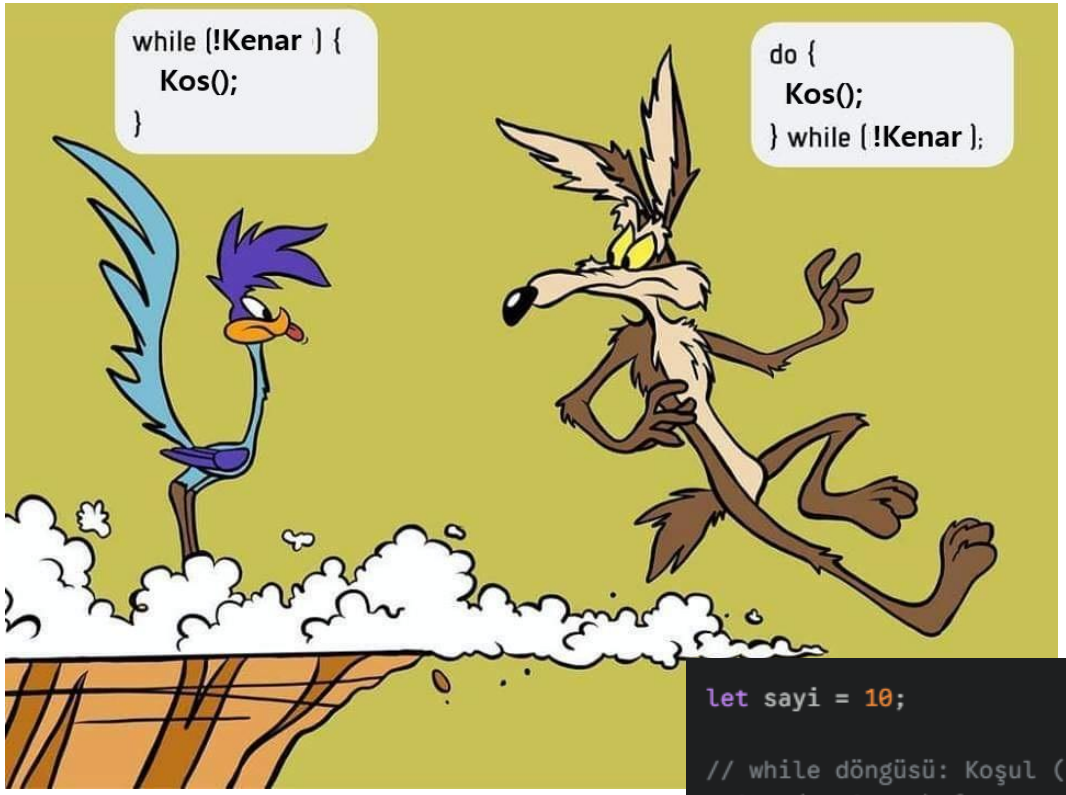
İpucu: Bu uygulama için **do...while** döngüsünü kullanınız.

KOD:

- js-react-ders-sunum-kodlari >> dongu_doWhile_uygulama.html
- js-react-ders-sunum-kodlari >> dongu_doWhile_uygulama.html

JavaScript

while / do..while farkı




```
let sayi = 10;

// while döngüsü: Koşul (10 < 5) yanlış olduğu için HİÇ çalışmaz.
while (sayi < 5) {
  console.log("while çalıştı");
}

// do...while döngüsü: Önce gövdeyi çalıştırır, sonra kontrol eder.
do {
  console.log("do...while çalıştı"); // Bu çıktı konsolda görünür!
} while (sayi < 5);
```

Döngüde kullanılan anahtar kelimeler

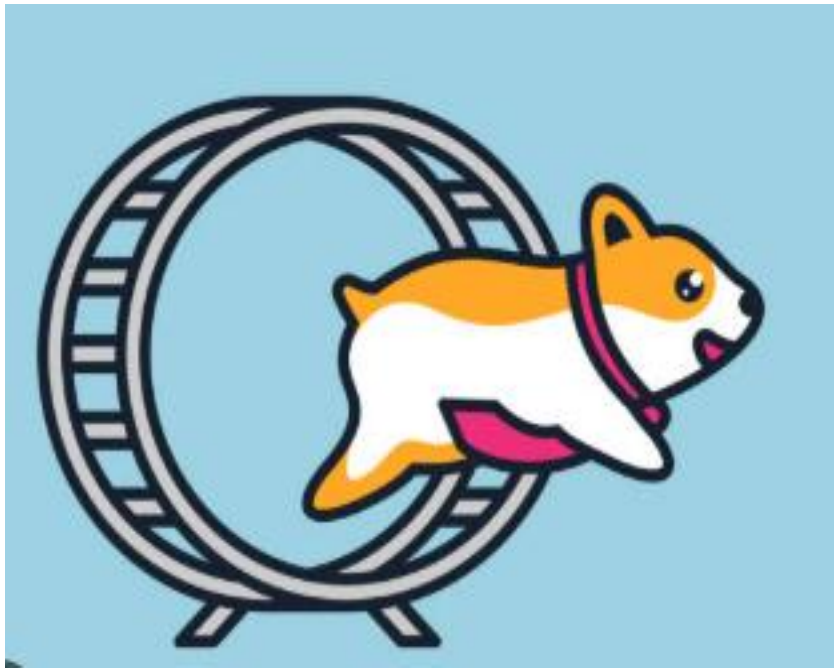
JavaScript

continue

Adımı Atla ve Devam Et

break

Döngüyü Kır ve Çık



Bütün döngülerde çalışır – for, while, do…while.

JavaScript

continue

continue komutu, döngünün o anki turunu (iteration) sonlandırır ve hemen bir sonraki tura geçer. **Döngü tamamen bitmez, sadece mevcut adımın altındaki kodlar o seferlik çalışmaz.**

break

break komutu ile karşılaşıldığında, döngü o anda durur ve döngüden tamamen çıkılır. **Döngünün geri kalan adımları çalıştırılmaz**.

Bütün döngülerde çalışır – for, while, do…while.

JavaScript

continue

break

1'den 5'e kadar sayalım ama 3 rakamını sevmediğimiz için onu atlayalım.

```
<script>
  for (let i = 1; i <= 5; i++) {
    if (i === 3) {
      document.write("bu değeri atladım - çevirmeye girmedi" + "<br>");
      continue; // i 3 olduğunda bu adımı atla ve sonraki adıma geç
    }
    document.write(i + "<br>"); // Her satırdan sonra boşluk bırakmak için
  }
</script>

<h3>Break Örneği</h3>
<script>
  for (let i = 1; i <= 5; i++) {
    if (i === 3) {
      document.write("bu değeri döngüyü kırar, gerisi gelmez" + "<br>");
      break; // i 3 olduğunda döngüyü sonlandır
    }
    document.write(i + "<br>"); // Her satırdan sonra boşluk bırakmak için
  }
```


KOD: js-react-ders-sunum-kodlari >> dongu_continue_break.html

JavaScript

break / continue

Komut	Ne Yapar?	Döngüye Ne Olur?
`break`	Döngüden hemen dışarı çıkar.	Döngü **tamamen biter.**
`continue`	Mevcut adımı yarıda keser.	Döngü **bir sonraki adımdan devam eder.**

Çalışma soruları

1. **Fibonacci Serisi**
   - while döngüsü ile, **ilk 10 Fibonacci sayısını** hesaplayıp ekrana yazdırın.
2. **Toplam Hesaplama**
   - Kullanıcıdan **sayılar girmesini** isteyin.
   - Kullanıcı 0 girene kadar sayı istemeye devam edin.
   - Girilen sayıların toplamını döngü sonunda console'a yazdırın.
3. **Sayı Tahmin Oyunu**
   - Bilgisayar 1–50 arasında rastgele bir sayı seçsin.
   - Kullanıcı tahminini girsin.
   - Kullanıcı doğru tahmin edene kadar do…while ile tekrar sorulsun.
   - Her yanlış tahminde "Daha büyük" veya "Daha küçük" uyarısı verilsin.

Çalışma soruları

4. **Asal Sayılar**
   - 1–50 arası **asal sayıları** bulan bir for döngüsü yazınız ve console'a yazdırınız
5. **Yıldız Piramidi**
   - for döngüsü kullanarak 5 satırlık bir **yıldız piramidi** oluşturunuz.

```
    *
   **
  ***
 ****
*****
```

6. **Toplam ve Ortalama Hesaplama**
   - Kullanıcıdan sayılar alın, kullanıcı -1 girene kadar devam edin.
   - Girilen sayıların **toplamını ve ortalamasını** döngü sonunda yazdırın.
7. **Kullanıcı Girişi Kontrolü (Şifre)**
   - Kullanıcıdan bir şifre girmesini isteyin.
   - Şifre "1234" olana kadar tekrar sorulsun.
   - Doğru şifre girildiğinde "Giriş Başarılı" mesajı verilsin.

Çalışma soruları

8. **Sayı Tahmin Oyunu (break)**
   - 1–50 arasında rastgele bir sayı seçilsin.
   - Kullanıcı tahmin etsin.
   - Tahmin doğruysa döngü bitsin.
   - Her yanlış tahminde “Daha büyük” veya “Daha küçük” mesajı verilsin.
9. **Pozitif Sayıların Toplamı (continue)**
   - Kullanıcıdan 5 sayı girmesi isteniyor.
   - Eğer sayı negatif ise **uyarı verilir ve toplam hesabına dahil edilmez** (continue).
   - Döngü sonunda yalnızca pozitif sayıların toplamı console’a yazdırılır.
10. **Çift Sayılar Listesi (continue)**
   - 1–20 arasındaki sayıları kontrol edin.
   - Eğer sayı **tek ise** ise atlayın.
   - Çift sayıları console’a yazdırın.

Diziler

- Birden fazla değeri tek bir değişken içinde depolamanızı sağlayan oldukça güçlü ve esnek yapılardır.
- JavaScript dizileri dinamiktir; yani boyutları kendiliğinden büyüyebilir.
- İçine farklı veri tiplerini (sayı, metin, nesne, hatta başka dizileri aynı anda koyabilirsiniz.

JavaScript

Dizi Tanımlama/Oluşturma

```
let dizi = [e1,e2,…,en]
```

```
let liste = new Array();
liste[0] = “deger1”;
liste[1] = 2;
```

```
let dizi = new
Array(e1,e2,…,en);
```

```
let dizin = new Array(n)
```

n→ dizi uzunluğu (length)
typeof → object

```
1  // Boş bir dizi
2  let meyveler = [];
3  // Elemanlı bir dizi
4  let arabalar = ["Tesla", "BMW", "Audi"];
5
6  let sayilar = new Array();
7  console.log(sayilar);
8  console.log(typeof sayilar);
9  sayilar[0] = 1;
10 sayilar[1] = 2;
11 sayilar[2] = "deger";
12
13 console.log(arabalar);
14 console.log(sayilar);
```

KOD: js-react-ders-sunum-kodlari >> dizi_olusturma.html

Elemanlara Erişme ve Değiştirme

- Diziler ilk indeks 0'dan başlar.

```
let arabalar = ["Tesla", "BMW", "Audi"];
```

- **Erişim:** arabalar[0] size "Tesla" değerini verir.
- **Güncelleme:** arabalar[1] = "Mercedes"; dediğinizde BMW artık Mercedes olur.
- **Uzunluk:** `arabalar.length` dizide kaç eleman olduğunu söyler.

Dizi elemenları

- Bir dizinin ilk elemanının indeks numarası her zaman **0**
- Son elemanının indeks numarası her zaman **(diziDegiskeni.length-1)**'dir.
- Dizi içerisindeki elemanlara erişmek, değerlerini değiştirmek ya da diziye yeni eleman eklemek için, **diziDegiskeni[index]** söz dizimi kullanılır.
- Diziye eleman eklemek istediğimizde **diziDegiskeni[index] = değer** ile atama yapılır.
- Eleman eklemede indeks kontrolü için fonksiyonların kullanımı daha doğru bir uygulamadır. Aksi durumda ara elemanlar 'undefined' olarak eklenir.
- Dizi elemanları farklı türden olabilir. (**KOD**: js-react-ders-sunum-kodlari >> dizi_eleman_turleri.html)

KOD: js-react-ders-sunum-kodlari >> dizi_elemanlarina_erisim.html

Dizi elemenları

Uygulama:

- Dersler dizisi ["mbp192", "mbp214", "mbp102", "mbp223", "mbp196"] tanımlayın,
- dizi sonuna indeks ile yeni " mbp205" dersini ekleyin.
- Diziye 10. eleman olarak " mbp333" dersini ekleyin
- Dizinin 10. ve 9. indeksteki elemanlarını ve elemanların türlerini yazdırın.
- Dizi elemanlarının değer ve her değerin türünü bir döngü ile yazdırın

KOD: js-react-ders-sunum-kodlari >> dizi_elemanlarina_erisim.html

For...in döngüsü

- Bir nesnenin (object) **sayılabilir özelliklerini** *(enumerable properties - numaralandırılabilir özellikler)* dönmek için kullanılır.
- Dizilerde for...in kullandığınızda, size elemanların kendisini değil, **indeks numaralarını** (string olarak) verir.

```
<script>
  const renkler = ["Kırmızı", "Mavi", "Yeşil"];

  for (let indeks in renkler) {
    document.write(indeks + ". indeks | anahtar veri türü: " + typeof indeks);
    document.write("<br>");
    document.write("değeri: " + renkler[indeks]); // Kırmızı, Mavi, Yeşil
    document.write("<br>");
  }
</script>
```

For...of

- Bir dizinin (veya string, Map, Set gibi yapıların) elemanlarını tek tek alıp işlem yapmanı sağlar.

“Dizinin içindeki her bir değeri sırayla bana ver”

```
<script>
  const sayilar = [5, 10, 15];
  let toplam = 0;

  for (const sayi of sayilar) {
    document.write(sayi + "<br>");
    toplam += sayi;
  }

  document.write("Toplam: " + toplam + "<br>");
</script>
```

!!!! KOD: js-react-ders-sunum-kodlari >> dizi_for_of.html

JavaScript

For...of

```
<script>
  document.write("<h3>Indeks verir</h3><br>");
  for (const index in sayilar) {
    document.write(index + "<br>");
    // index verir
  }

  document.write("<h3>Değerleri verir</h3><br>");
  for (const value of sayilar) {
    document.write(value + "<br>");
  } // değeri verir
</script>
```

!!!! KOD: js-react-ders-sunum-kodlari >> dizi_for_of.html

Soru

- Aşağıda verilen dizide öğrenci adı, vize ve final notları verilmiştir. Vize %40, final %60 etkilidir. Her öğrencinin dönem sonu notunu hesaplayınız

```
let notlar = [
  ["Ali", 80, 90],
  ["Ayşe", 70, 85],
  ["Mehmet", 60, 75],
];
```

!!!! KOD: js-react-ders-sunum-kodlari >> dizi_cok_boyutlu_soru.html

Sık Kullanılan Dizi Metotları - 1

.pop()

- Dizinin **sonundaki** elemanı siler ve sildiği elemanın değerini geriye döndürür.
- **Söz dizimi:** `diziDegiskeni.pop()`

.shift()

- dizi içerisindeki **ilk elemanı** (0. indeksteki eleman) siler ve sildiği elemanın değerini geriye döndürür.
- Bu işlem dizinin uzunluğunu (length) bir azaltır.
- **Söz dizimi:** `diziDegiskeni.shift()`

.push()

- Bir dizinin sonuna bir ya da daha fazla eleman eklemek için kullanılır.
- Bu metot geriye dizinin yeni oluşan uzunluğunu döndürür.
- **Söz dizimi:** `diziDegiskeni.push(eleman1, eleman2 ... elemanN)`

.unshift()

- Bir dizinin **başına** bir ya da daha fazla eleman eklemek için kullanılır.
- Bu metot, ekleme işleminden sonra dizinin **yeni oluşan uzunluğunu** (length) geriye döndürür.
- **Söz dizimi:** `diziDegiskeni.unshift(eleman1, eleman2 ... elemanN)`

KOD: js-react-ders-sunum-kodlari >> dizi_fonksiyonlari-1.html

Sık Kullanılan Dizi Metotları - 2

JavaScript

.reserse()

- Dizi içerisindeki sıralamayı ters çevirir. Yani ilk baştaki eleman en sona, en sondaki eleman ilk başa getirilir.
- Bu metot geriye yeni oluşturulan dizi nesnesini döndürür.
- Dizi üstünde kalıcı bir işlem yapar.
- **Söz dizimi:** `diziDegiskeni.reverse()`

.join()

- Dizideki tüm elemanları birleştirerek **tek bir metin (string)** oluşturmanızı sağlar.
- **Söz dizimi:** `dizi.join(ayirici);`
- **Boş Değerler:** Eğer dizide null veya undefined elemanlar varsa, join() bunları boş bir string ("") olarak işler.
- **Sayılar**: Dizide sayılar olsa bile, join() hepsini string'e çevirip birleştirir.

.split()

- Bir **metni (string)**, belirttiğiniz bir ayırıcıya (separator) göre parçalara böler ve bu parçalardan oluşan **yeni bir dizi (array)** döndürür.
- Join() tersi
- **Söz dizimi:** `metin.split(ayirici, limit);`
- **ayirici:** Metnin nereden bölüneceğini belirler (boşluk, virgül, tire vb.).
- **limit (Opsiyonel):** Oluşturulacak dizinin maksimum eleman sayısını belirler.

KOD: js-react-ders-sunum-kodlari >> dizi_fonksiyonlari-4.html

Sık Kullanılan Dizi Metotları - 2

JavaScript

.slice()

- Bir dizinin belirli bir kısmını kopyalayarak **yeni bir dizi** oluşturmak için kullanılır.
- orijinal dizide hiçbir değişiklik yapmamasıdır (**immutable**).
- **Söz Dizimi:**

```
dizi.slice(baslangic, bitis);
```

- **baslangıç:** Kopyalamanın başlayacağı indeks (dahildir).
- **bitiş:** Kopyalamanın biteceği indeks (**dahil değildir**). Eğer belirtilmezse dizinin sonuna kadar gider.

.splice()

- Aynı anda hem **eleman silebilir**, hem **ekleyebilir**, hem de **güncelleyebilir**
  - Sadece Eleman Silme
  - Eleman Ekleme (Hiç Silmeden)
  - Eleman Değiştirme (Güncelleme)
- Orijinal diziyi **doğrudan değiştirir** (mutate eder).
- **Söz dizimi:**

```
dizi.splice(başlangıç
indeksi, silinecekAdet,
eklenecekEleman1,
eklenecekEleman2, ...);
```

!!!! KOD: js-react-ders-sunum-kodlari >> dizi_fonksiyonlari-2.html

Sık Kullanılan Dizi Metotları - 3

```
diziDegiskeni.sort()
diziDegiskeni.sort(compareNumber)
```

- Sıralama yaparken dizi içerisindeki değerlerin **Unicode** karşılıklarını kullanır.
- Sıralama yapılırken, veri türü farkı olmadan tüm veriler karakter bazında birbirleriyle karşılaştırılır.

.sort()

- Sıralama büyük/küçük harf duyarlıdır.
- Sıralama artan sırada gerçekleştirilir.
- Karşılaştırma sonucunda eşit olarak kabul edilen öğeler art arda yerleştirilirler.

KOD: js-react-ders-sunum-kodlari >> dizi_fonksiyonlari-3.html

forEach

- Dizinin her elemanı üzerinde tek tek işlem yapmanı sağlayan bir metottur.
- **kontrol gücü düşüktür**—özellikle **break** veya **continue kullanamamak** önemli bir sınırlamadır.
- Yazımına dikkat – süslü + normal parantez

```
dizi.forEach(function(eleman, index, array) {
  // yapılacak işlem
});
```

eleman → dizideki değer
index → o değerin sırası
array → tüm dizi (genelde az kullanılır)

forEach

JavaScript

```
<h2>Kullanım Örneği 1</h2>
<script>
  const kullanicilar = ["Ali", "Ayşe", "Mehmet"];

  kullanicilar.forEach((isim, index) => {
    document.write(`${index + 1}. Kullanıcı: ${isim} <br>`);
  });
  // Çıktı:
  // 1. Kullanıcı: Ali
  // 2. Kullanıcı: Ayşe
  // 3. Kullanıcı: Mehmet
</script>
<h2>Kullanım Örneği 2</h2>
<script>
  let sayilar = [1, 2, 3, 4, 5];

  sayilar.forEach((sayi) => {
    document.write(sayi * 2);
    document.write("<br>");
  });
</script>
```

!!!! **KOD**: js-react-ders-sunum-kodlari >> dizi_foreach.html

JavaScript

For...Of / Foreach farkı

Foreach

- Bir dizinin **her elemanı için otomatik çalışan** bir metottur.
- **break ❌ kullanamazsın**
- **continue ❌ kullanamazsın**
- Döngüyü erken durduramazsın . Kontrol sende değil, JavaScript'te.

For...Of

- Diziler üzerinde dolaşmak için kullanılan **gerçek bir döngüdür**.
- **break ✔ kullanabilirsin**
- **continue ✔ kullanabilirsin**
- Döngüyü istediğin yerde kesebilirsin.

!!!! KOD: js-react-ders-sunum-kodlari >> dizi_kiyaslama.html

For...Of / Foreach farkı

Foreach

```
<script>
    const arr = [1, 2, 3, 4];

    arr.forEach((item) => {
    if (item === 3) {
        break; // HATA verir
    }
  });
</script>
```

For...Of

```
<script>
  for (const item of arr) {
    if (item === 3) {
      break;
    }
    document.write(item + "<br>");
  }
</script>
```

!!!! KOD: js-react-ders-sunum-kodlari >> dizi_kiyaslama.html

Çok boyutlu diziler

- **Çok boyutlu diziler (multi-dimensional arrays)** aslında “dizi içinde dizi” mantığıdır.

```
<script>
  let dizi = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
  ];

  document.write(dizi[1][2]); //6
</script>
```

Soru

Aşağıda verilen dizi içerisinde gezerek:

- Sayıları ekrana yazdıran,
- **50'den büyük ilk sayıyı bulduğunda döngüyü durduran (break)**
- JavaScript kodunu **for...of döngüsü kullanarak** yazınız.

```
const sayilar = [12, 25, 47, 52, 64, 30];
```

Soru

JavaScript

Aşağıda verilen dizide öğrenci adı, vize ve final notları verilmiştir. Vize %40, final %60 etkilidir. Her öğrencinin dönem sonu notunu hesaplayınız.

- Eğer öğrenci final notu 0 ise, 'dersten kaldı' yazınız.
- Eğer öğrenci adı bulunmuyor ise o satırı atlayınız.
- Kodun sonunda kaç öğrenci atlandığını da yazdırınız

```
let notlar = [
  ["Ali", 80, 90],
  ["Ayşe", 70, 85],
  ["", 70, 85],
  ["Mehmet", 60, 75],
  ["Volkan", 60, 0],
];
```

!!!! KOD: js-react-ders-sunum-kodlari >> dizi_cok_boyutlu_soru-2.html

Soru

Aşağıda verilen dizi içerisinde gezerek:

- Sayıları ekrana yazdıran,
- **50'den büyük ilk sayıyı bulduğunda döngüyü durduran (break)**
- JavaScript kodunu **for...of döngüsü kullanarak** yazınız.

```
const sayilar = [12, 25, 47, 52, 64, 30];
```

Çalışma soruları

Dizi içinde dolaşarak:

- **Çift sayıları atla (continue)**
- **40'tan büyük ilk sayıda dur (break)**
- Sadece uygun sayıları yazdır

```
const sayilar = [10, 15, 22, 35, 41, 18];
```

Akış

JavaScript

VİZE KONULARI

- **Javascript Temelleri**
- **Değişkenler ve Veri türleri**
- **Tür dönüşümü**
- **Operatorler**
- **Kontrol İfadeleri**
- **Temel createElement ve appandChild ile DOM uygulaması**
- **Döngüler ve İterasyon**
  - For
  - Do while
  - While
  - Break / Continue
- **Diziler**
  - Dizi fonksiyonları
  - Foreach
  - For ..in
  - For ..of
  - Çok boyutlu diziler

VİZE SONRASI

- **Diziler**
  - every()
  - some()
  - filter()
  - map()
- Fonksiyonlar
- Callback functions
  - Sayı sort()
- Hata Yönetimi - Error Handling
- Debugging
- DOM Manipulation (3)
- Form Validation
- Using APIs
- React